

Plataforma de Serviços de Rede Baseada em containers

Daniel Taveira - a44319

Trabalho realizado sob a orientação de

Prof. Nuno Rodrigues

Prof. Eduardo Costa

Licenciatura em Engenharia Informática

2025-2026

Plataforma de Serviços de Rede Baseada em containers

Relatório da UC de Projeto
Licenciatura em Engenharia Informática
Escola Superior de Tecnologia e Gestão

Daniel Taveira - a44319

2025-2026

A Escola Superior de Tecnologia e de Gestão não se responsabiliza pelas opiniões expressas neste relatório.

Declaro que o trabalho descrito neste relatório é da minha autoria e é da minha vontade que o mesmo seja submetido a avaliação.

Daniel Taveira - a44319

Agradecimentos

Em primeiro lugar, gostaria de expressar a minha profunda gratidão ao meu orientador, Prof. Nuno Rodrigues, pela disponibilidade, partilha de conhecimento e orientação constante ao longo do desenvolvimento deste projeto. O seu apoio técnico e metodológico foi fundamental para a concretização deste trabalho.

Um agradecimento muito especial e sentido à minha mãe, pelo apoio incondicional, pelo sacrifício e pela força que sempre me transmitiu ao longo de todo o meu percurso académico e pessoal. Sem ti, nada disto seria possível.

Por fim, à minha namorada, pela infinita paciência, compreensão e por estar sempre ao meu lado nos momentos de maior exigência e stress. O teu encorajamento diário foi a minha maior motivação para chegar até aqui.

Resumo

O presente projeto tem como objetivo principal a conceção e implementação de uma plataforma de serviços de rede gerida de forma reprodutível, recorrendo exclusivamente a ferramentas *open source*. A infraestrutura desenvolvida engloba serviços fundamentais para uma rede de pequena a média dimensão, tais como servidor DNS, *Reverse Proxy* HTTPS, serviços *web*, e-mail, servidor de ficheiros e VPN.

Em contraste com abordagens monolíticas tradicionais, todos os serviços são implementados em *containers* (Docker/Podman) através de configuração declarativa, garantindo o versionamento e a reprodutibilidade do ambiente. O trabalho foca-se na aplicação rigorosa de boas práticas de segurança, incluindo o princípio do mínimo privilégio, *hardening* de serviços e encriptação TLS. Adicionalmente, o projeto integra mecanismos avançados de observabilidade (para métricas e *logs* centralizados) e a orquestração de *deployments* através de ferramentas como Ansible e *pipelines* de CI/CD. O resultado final demonstra como um pequeno *data center* pode ser gerido de forma automatizada, ágil e resiliente a falhas.

Palavras-chave: *Containers*, Orquestração, Serviços de Rede, CI/CD, Observabilidade, *Open Source*.

Abstract

The main objective of this project is the design and implementation of a network services platform managed in a reproducible way, using exclusively open-source tools. The developed infrastructure encompasses fundamental services for a small to medium-sized network, such as a DNS server, HTTPS Reverse Proxy, web services, e-mail, file server, and a VPN.

In contrast to traditional monolithic approaches, all services are implemented in containers (Docker/Podman) through declarative configuration, ensuring environment versioning and reproducibility. The work focuses on the strict application of security best practices, including the principle of least privilege, service hardening, and TLS encryption. Additionally, the project integrates advanced observability mechanisms (for metrics and centralized logs) and deployment orchestration using tools like Ansible and CI/CD pipelines. The final result demonstrates how a small data center can be managed in an automated, agile, and fault-resilient manner.

Keywords: Containers, Orchestration, Network Services, CI/CD, Observability, Open Source.

Índice

1	Introdução	1
1.1	Contextualização e Motivação	1
1.2	Objetivos	2
1.3	Metodologia	3
1.4	Estrutura do Documento	3
2	Estado da Arte	5
2.1	Virtualização e <i>Containers</i>	5
2.1.1	Orquestração: Docker Compose versus Kubernetes	6
2.2	Ferramentas de Automação e Infraestrutura como Código	6
2.3	Serviços de Rede: Alternativas e Justificações	7
2.3.1	Resolução de Nomes: Pi-hole vs. BIND9	7
2.3.2	Reverse Proxy: Nginx vs. Apache e Traefik	7
2.3.3	Redes Privadas Virtuais: WireGuard vs. OpenVPN	7
2.4	Observabilidade: Prometheus e Loki vs. ELK Stack	7
2.5	CI/CD com GitLab	8
3	Levantamento das Necessidades Funcionais	9
3.1	Serviços de Rede a Disponibilizar	9
3.2	Requisitos Técnicos e de Arquitetura	10
3.3	Ambiente Base: Virtualização e Sistema Operativo	12
3.3.1	Esquema Gráfico da Arquitetura Física	12

3.3.2	Configuração de Rede do Servidor	14
3.3.3	Acesso Externo: Mapeamento em DMZ e Dynamic DNS (DDNS)	14
4	Arquitetura e Implementação	15
4.1	Arquitetura Geral da Plataforma	15
4.2	Configuração do Ambiente Base (Ubuntu Server)	16
4.2.1	Instalação e Configuração Inicial	16
4.2.2	Configuração de IP Estático com Netplan	17
4.2.3	Problema de <i>Bridging</i> no VMware	18
4.2.4	Resolução Crítica de Esgotamento de Espaço em Disco (LVM)	20
4.3	Automação com Ansible	21
4.3.1	Criptografia e Gestão de Segredos com Ansible Vault	22
4.4	Implementação dos Serviços via Docker Compose	23
4.4.1	Servidor DNS: Pi-hole	24
4.4.2	Servidor Web e <i>Reverse Proxy</i> : Nginx	25
4.4.3	Cloud Corporativa e Servidor de E-mail	26
4.4.4	VPN: WireGuard	27
4.5	Implementação de CI/CD com GitLab	31
4.6	Observabilidade Avançada: Múltiplos Dashboards no Grafana	33
4.6.1	Resolução de Leitura de Logs (Docker Socket no Promtail)	34
4.7	Síntese das Configurações de Segurança Aplicadas	37
5	Testes e Avaliação	39
5.1	Metodologia de Testes	39
5.2	Teste 1: Resolução DNS e Filtragem Ativa (Pi-hole)	40
5.3	Teste 2: Reverse Proxy e Isolamento Read-Only (Nginx)	41
5.4	Teste 3: Acesso Remoto VPN e Dynamic DNS (WireGuard)	42
5.5	Teste 4: Orquestração CI/CD e Proteção Vault	42
5.6	Teste 5: Nuvem Corporativa e E-mail (Nextcloud e Mailserver)	43
5.7	Teste 6: Centralização de Logs e Métricas (Loki, Prometheus e Grafana)	43

5.8	Análise Crítica dos Resultados	44
5.8.1	Balanco de Objetivos Cumpridos	44
5.8.2	O que Poderia ter sido Feito de Forma Diferente (Reflexão)	44
6	Conclusões	47
6.1	Síntese do Trabalho Realizado	47
6.2	Aprendizagens e Contribuições	48
6.3	Comparação com Trabalhos Relacionados	48
6.4	Trabalho Futuro	49
6.5	Considerações Finais	50
A	Proposta Original do Projeto	A1
B	Listagens de Código Fonte e Configurações	B1
B.1	Docker Compose da Plataforma (docker-compose.yml)	B1
B.2	Ansible Playbook de Setup (setup_docker.yml)	B6
B.3	Ansible Playbook de Deploy (deploy_servicos.yml)	B7
B.4	GitLab CI/CD Pipeline (.gitlab-ci.yml)	B8

Lista de Tabelas

2.1	Comparação entre Máquinas Virtuais e <i>Containers</i>	6
3.1	Serviços Implementados na Stack e Ferramentas Seleccionadas	11
4.1	Medidas de Segurança Aplicadas na Plataforma	37

Lista de Figuras

3.1	Esquema gráfico da arquitetura física, fronteira de rede e endereçamento IP do sistema.	13
4.1	Esquema conceptual da arquitetura da plataforma estruturado em 3 camadas abstratas.	16
4.2	Verificação do endereço IP e acesso SSH inicial ao Ubuntu Server 24.04 LTS.	17
4.3	VMware Virtual Network Editor com o adaptador Realtek selecionado manualmente para <code>VMnet0</code> (modo <i>Bridged</i>).	20
4.4	Estrutura do repositório e ficheiro de inventário do Ansible (<code>inventory.ini</code>).	21
4.5	Execução bem-sucedida do <i>Playbook</i> <code>setup_docker.yml</code> — <i>Play Recap</i> com <code>failed=0</code>	22
4.6	Visualização do bloco criptografado AES-256 após aplicação do Ansible Vault.	23
4.7	Estado operacional dos contentores ativos na plataforma.	23
4.8	Interface de administração do servidor DNS (Pi-hole) a correr in <i>container</i> .	25
4.9	Página Web servida pelo Nginx através de um Volume Docker em modo <i>Read-Only</i>	26
4.10	QR code gerado pelo <i>container</i> WireGuard para configuração automática do cliente móvel (<i>peer1</i>).	28
4.11	Ligação VPN WireGuard estabelecida com sucesso no dispositivo móvel (estado <i>Active</i>).	30
4.12	Registo bem-sucedido do GitLab Runner no servidor Ubuntu.	31

4.13	<i>Runner</i> ativo e em comunicação com o repositório GitLab (estado <i>online</i>).	31
4.14	Execução bem-sucedida da <i>pipeline</i> de CI/CD — <i>deploy</i> automático da infraestrutura a partir de um <i>commit</i>	33
4.15	Interface de <i>targets</i> do Prometheus com ambos os serviços no estado <i>UP</i> . .	33
4.16	Configuração do Prometheus como <i>Data Source</i> no Grafana, com estado de ligação confirmado.	35
4.17	Logs centralizados no Grafana Loki demonstrando a captura de eventos de containers em tempo real.	36
4.18	Painel <i>Node Exporter Full</i> no Grafana a exibir métricas do servidor em tempo real.	36
5.1	Monitorização de consultas DNS bloqueadas e processadas no painel Pi-hole.	40

Capítulo 1

Introdução

A administração de sistemas e redes vive hoje uma transformação profunda, impulsionada por exigências cada vez maiores de disponibilidade, escalabilidade e rastreabilidade. Gerir infraestruturas à mão, serviço a serviço, tornou-se um risco operacional que muitas organizações já não podem assumir. A necessidade de implementar plataformas de forma rápida, reprodutível e resistente a falhas humanas tem forçado a transição para tecnologias de *containerização* e para a filosofia de *Infrastructure as Code* (IaC) [1].

1.1 Contextualização e Motivação

Em infraestruturas de pequena ou média dimensão — um cenário frequente em laboratórios de investigação, PMEs ou ambientes de ensino —, a gestão manual de serviços críticos como DNS, *reverse proxies*, VPNs ou servidores de ficheiros torna-se rapidamente insustentável. Cada intervenção direta no sistema é um ponto de falha: ficheiros de configuração modificados sem rasto, dependências instaladas que não figuram em documentação ou serviços repostos com parâmetros divergentes dos originais são erros comuns em ambientes que carecem de automação.

A motivação deste projeto surgiu precisamente desta realidade. Durante a implementação, a ausência de uma base de automação agravou os tempos de recuperação após incidentes. Um exemplo claro foi a simples reinicialização do *router* da rede local, que

derrubou a conectividade da máquina virtual (VM) e exigiu uma intervenção manual de diagnóstico e reconfiguração. Num ambiente automatizado, esta intervenção seria evitável ou resolvida em instantes. Este incidente, que será analisado detalhadamente no Capítulo 4, serve como prova da relevância prática dos objetivos deste trabalho.

O paradigma de *containerização*, através de ferramentas como o Docker, resolve estes problemas ao encapsular aplicações e serviços em unidades portáteis e isoladas. As dependências são declaradas de forma explícita, assegurando que o ambiente de execução é consistente, independentemente da máquina física hospedeira [2]. Quando combinamos esta portabilidade com a automação via Ansible e *pipelines* de CI/CD, garantimos que qualquer alteração à infraestrutura é auditada, testada e aplicada de forma controlada, evitando o "desvio de configuração" (*configuration drift*) [3].

1.2 Objetivos

O objetivo central deste trabalho é construir, configurar e documentar uma plataforma de serviços de rede totalmente baseada em *containers*, orquestrada com Docker Compose, focada em segurança, observabilidade e CI/CD.

De forma específica, os objetivos operacionais definidos são:

- **Implementar serviços de rede em *containers*:** Servidor DNS (Pi-hole), *Reverse Proxy* HTTPS (Nginx), servidor Web (Nginx), VPN (WireGuard), Servidor de E-mail (Docker-Mailserver), servidor de ficheiros (Nextcloud) e *stack* de monitorização (Prometheus, Grafana, Loki).
- **Garantir reprodutibilidade total:** A infraestrutura deve ser recriável a partir do repositório Git sem intervenção manual, seguindo o princípio da *Single Source of Truth* [1].
- **Aplicar práticas rigorosas de segurança:** Utilização de volumes *read-only*, segmentação de redes internas Docker, encriptação de segredos com *Ansible Vault* e reforço de tráfego via TLS/WireGuard.

- **Automatizar o *deploy* com CI/CD:** Qualquer *commit* na *branch* principal deve despoletar automaticamente o *deploy* da infraestrutura, assegurando consistência entre o código e o ambiente de produção.
- **Integrar observabilidade centralizada:** Implementação de recolha de métricas de *hardware* e logs centralizados, com visualização em painéis Grafana para monitorização proativa.
- **Documentar procedimentos de *troubleshooting*:** Registo detalhado dos desafios técnicos e erros encontrados durante o projeto, demonstrando o raciocínio aplicado na sua resolução.

1.3 Metodologia

A metodologia adotada divide-se em três eixos principais. Primeiro, uma estratégia incremental: os serviços foram implementados por camadas, começando pela infraestrutura base (SO e Docker), progredindo para os serviços de rede (DNS, Web, VPN, E-mail) e culminando na observabilidade e automação. Esta estratificação permitiu validar cada camada antes de avançar para a próxima.

Segundo, o versionamento contínuo: toda a configuração foi mantida em Git, permitindo o rastreio de alterações e a reversão imediata de configurações problemáticas. Por fim, uma documentação ativa: cada obstáculo técnico foi registado no momento da ocorrência, preservando o contexto necessário para que este relatório sirva não apenas como descrição, mas como *runbook* de referência para futuras intervenções.

1.4 Estrutura do Documento

O relatório organiza-se da seguinte forma:

- **Capítulo 2 — Estado da Arte:** análise das tecnologias escolhidas face às alternativas de mercado.

- **Capítulo 3 — Levantamento das Necessidades:** requisitos funcionais e análise da arquitetura base.
- **Capítulo 4 — Arquitetura e Implementação:** detalhe técnico da montagem da infraestrutura e resolução de problemas.
- **Capítulo 5 — Testes e Avaliação:** validação prática dos serviços e discussão dos resultados.
- **Capítulo 6 — Conclusões:** síntese do trabalho e propostas de evolução futura.

Capítulo 2

Estado da Arte

Este capítulo analisa as tecnologias fundamentais que suportam a plataforma, fundamentando as opções tomadas face ao estado da arte atual. O objetivo é contextualizar cada componente, comparando-o com as alternativas comuns de mercado para justificar as escolhas que tornam esta infraestrutura eficiente, segura e reproduzível.

2.1 Virtualização e *Containers*

Historicamente, a virtualização baseada em *hypervisors* (como VMware ESXi, Hyper-V ou KVM) revolucionou a partilha de recursos físicos [4]. Contudo, o custo computacional de replicar um sistema operativo completo para cada máquina virtual (VM) torna-se excessivo em ambientes onde a densidade de serviços é a prioridade.

A tecnologia de *containers* inverte este paradigma. Em vez de virtualizar o *hardware*, isolam-se processos ao nível do sistema operativo, partilhando o mesmo *kernel* do *host* através de *namespaces* e *cgroups* do Linux [2].

2.1.1 Orquestração: Docker Compose versus Kubernetes

Para orquestrar os serviços, avaliou-se o Kubernetes face ao Docker Compose. Embora o Kubernetes seja a referência industrial para ambientes de larga escala, a sua implementação num projeto desta dimensão introduz uma complexidade operacional e uma sobrecarga de recursos (*control plane*) injustificadas. Optou-se pelo **Docker Compose** pela sua natureza declarativa e simplicidade, permitindo definir toda a topologia num ficheiro YAML único, o que garante a reprodutibilidade com uma pegada de memória mínima.

Tabela 2.1: Comparação entre Máquinas Virtuais e *Containers*

Característica	Máquina Virtual (VM)	<i>Container</i>
Isolamento	<i>Hardware</i> virtualizado completo	<i>Namespaces</i> e <i>cgroups</i> do <i>kernel</i>
Sistema Operativo	Independente por VM	Partilhado com o <i>host</i>
Tempo de arranque	Minutos	Segundos
Consumo de RAM	Vários GB por VM	Dezenas de MB por <i>container</i>

2.2 Ferramentas de Automação e Infraestrutura como Código

A filosofia de *Infrastructure as Code* (IaC) dita que o estado da infraestrutura deve ser descrito em ficheiros versionados [1]. O **Ansible** foi a ferramenta escolhida para aplicar este conceito. Enquanto alternativas como o *Puppet* ou *Chef* requerem a instalação de agentes nas máquinas-alvo, o Ansible adota uma abordagem *agentless*, comunicando apenas por SSH. A sua característica mais valiosa é a **idempotência**: o mesmo *playbook* pode ser executado vezes sem conta, garantindo que o servidor se mantém sempre no estado definido, independentemente da configuração inicial [9].

2.3 Serviços de Rede: Alternativas e Justificações

2.3.1 Resolução de Nomes: Pi-hole vs. BIND9

Embora o **BIND9** seja a norma em servidores de zona complexos, a sua gestão exige configurações densas de ficheiros de texto. O **Pi-hole** foi selecionado por oferecer as funções de servidor DNS com a vantagem adicional de filtragem de tráfego publicitário e uma interface web que simplifica a monitorização de consultas em tempo real, tornando a gestão muito mais ágil para uma infraestrutura média [11].

2.3.2 Reverse Proxy: Nginx vs. Apache e Traefik

O **Nginx** superou as alternativas (Apache e Traefik) pelo seu equilíbrio entre performance e simplicidade. O Apache apresenta um consumo de recursos superior devido ao seu modelo de processos, enquanto o Traefik, apesar da integração dinâmica com Docker, possui uma curva de aprendizagem mais acentuada. O Nginx destaca-se pela leveza e pela facilidade na configuração de blocos de proxy e terminação TLS [12].

2.3.3 Redes Privadas Virtuais: WireGuard vs. OpenVPN

O **WireGuard** foi escolhido por ser o atual estado da arte. Protocolos como o **OpenVPN** baseiam-se em *user space* e código extenso, o que dificulta auditorias de segurança e aumenta a latência. O WireGuard, integrado no *kernel* Linux, oferece uma latência reduzida, criptografia moderna (Curve25519) e uma base de código drasticamente menor, facilitando a verificação de segurança [15].

2.4 Observabilidade: Prometheus e Loki vs. ELK Stack

Na área da monitorização, o **ELK Stack** (Elasticsearch, Logstash, Kibana) é o padrão para grandes volumes de dados, mas o seu consumo de RAM (Java-based) é proibitivo

para o hardware deste projeto. A solução implementada — **Prometheus** (métricas) com **Loki** (logs) e **Grafana** (visualização) — permite recolher toda a telemetria necessária com uma fração do custo computacional. O Loki segue a filosofia de *labels* do Prometheus, evitando a indexação pesada de texto e mantendo o servidor Ubuntu estável e responsivo [13].

2.5 CI/CD com GitLab

A automação do ciclo de vida da plataforma é assegurada pelo **GitLab CI/CD** [14]. A opção por um *GitLab Runner* local (*self-hosted*) é estratégica: permite que o servidor interpele o GitLab para recolher *jobs* de deploy, eliminando a necessidade de expor APIs críticas de gestão à Internet pública, mantendo a superfície de ataque reduzida.

Capítulo 3

Levantamento das Necessidades Funcionais

Com base no guião do projeto proposto pela coordenação de Engenharia Informática da ESTiG, o objetivo central consiste na criação de um *data center* de serviços de rede gerido de forma estritamente reproduzível, recorrendo exclusivamente a ferramentas *open source*. As necessidades do projeto dividem-se em duas categorias principais: os serviços a disponibilizar (requisitos funcionais) e os requisitos técnicos e de segurança da arquitetura (requisitos não-funcionais).

3.1 Serviços de Rede a Disponibilizar

A infraestrutura contempla um ecossistema de serviços essenciais para uma rede empresarial de pequena ou média dimensão. De forma detalhada, os requisitos funcionais integrados assentam na seguinte especificação:

- **Servidor DNS Recursivo e Autoritativo:** Resolução de nomes para os hosts e serviços da rede interna; filtragem ativa de domínios publicitários e maliciosos; console gráfica de auditoria acessível por HTTP.

- **Reverse Proxy HTTPS:** Ponto unificado de entrada na rede para encaminhamento dinâmico de tráfego HTTP/HTTPS com base no subcaminho do pedido e terminação TLS.
- **Servidor Web Estático:** Disponibilização de um painel institucional de estado da infraestrutura através do Nginx com injeção de volumes em modo *read-only* para prevenção de desfiguração (*defacement*).
- **Servidor de Ficheiros Corporativo (Nextcloud):** Mecanismo de armazenamento centralizado em nuvem privada que garante a persistência de dados dos utilizadores de forma isolada, resiliente a falhas e controlada por volumes permanentes.
- **Servidor de E-mail Independente (Docker-Mailserver):** Solução completa de comunicações eletrónicas capaz de gerir caixas de correio locais através dos protocolos padrão SMTP e IMAP, incorporando políticas de segurança nativas (Fail2ban e SpamAssassin).
- **Acesso Remoto Seguro (WireGuard VPN):** Estabelecimento de um túnel cifrado à rede interna a partir de redes exteriores WAN, utilizando obrigatoriamente um mecanismo de *Dynamic DNS* para contornar a volatilidade do IP público do router doméstico.
- **Observabilidade e Logs Centralizados:** Recolha contínua de métricas de telemetria do host, monitorização do ciclo de vida dos contentores e centralização de logs estruturados (Prometheus, Loki, Promtail e Grafana).

A Tabela 3.1 apresenta detalhadamente os serviços operacionais implementados e as ferramentas selecionadas.

3.2 Requisitos Técnicos e de Arquitetura

Para além do correto funcionamento dos serviços, a plataforma respeita rigorosos critérios de engenharia e gestão declarativa:

Tabela 3.1: Serviços Implementados na Stack e Ferramentas Seleccionadas

Serviço	Ferramenta	Justificação Técnica
Servidor DNS	Pi-hole	Servidor recursivo/autoritativo com interface web e bloqueio de domínios a nível de rede [11].
<i>Reverse Proxy</i>	Nginx	Alto desempenho, baixo consumo de RAM, arquitetura assíncrona orientada a eventos [12].
Servidor Web Estático	Nginx	Distribuição de painel HTML institucional via volumes persistentes com mínimo overhead computacional.
Servidor de Ficheiros	Nextcloud	Plataforma líder para nuvem privada corporativa com isolamento persistente de dados.
Servidor de E-mail	Docker-Mailserver	Solução compacta e blindada (Postfix/Dovecot) com Fail2ban ativo para proteção da infraestrutura.
VPN Remota	Wire-Guard	Protocolo moderno de alto rendimento no kernel Linux, criptografia assimétrica e chaves curtas.
Recolha de Métricas	Prometheus	Base de dados temporal baseada em modelo pull com armazenamento otimizado em séries temporais [16].
Agregação de Logs	Grafana Loki	Agregação leve de logs Docker baseada em metadados labels, evitando a sobrecarga de memória RAM da stack ELK.
Visualização Unificada	Grafana	Painéis analíticos interativos com provisionamento automático de fontes de dados na mesma interface [17].

- **Containerização Estruturada:** Todos os serviços correm isolados em *containers* independentes via `docker-compose.yml`, garantindo portabilidade total.
- **Segurança Ativa (*Hardening*):** Aplicação do princípio do mínimo privilégio em volumes (`:ro`), isolamento de redes Docker do tipo bridge, cifragem obrigatória de tráfego e proteção criptográfica de segredos com Ansible Vault.
- **Pipeline de CI/CD Automation:** Automação baseada em GitLab Runner local para efetuar o deploy controlado da infraestrutura a cada modificação submetida ao repositório Git.

3.3 Ambiente Base: Virtualização e Sistema Operativo

Para suportar a plataforma e garantir o isolamento do *host* físico, a infraestrutura base foi implementada num ambiente virtualizado simulando um datacenter privado doméstico.

3.3.1 Esquema Gráfico da Arquitetura Física

A topologia física do sistema e o respetivo endereçamento IP estão representados de forma gráfica na Figura 3.1.

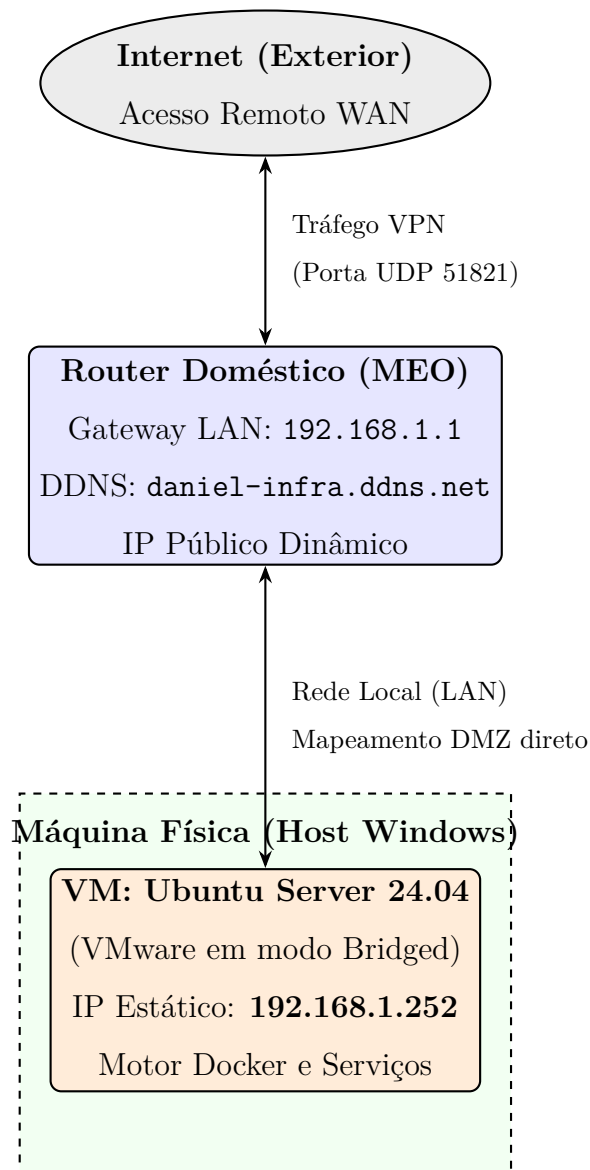


Figura 3.1: Esquema gráfico da arquitetura física, fronteira de rede e endereçamento IP do sistema.

A infraestrutura assenta na seguinte segmentação de hardware e virtualização:

- **Virtualização (VMware Workstation):** O hypervisor local VMware Workstation garante estabilidade e controlo granular sobre as placas de rede virtuais [4]. O modo *Bridged* foi selecionado para que a VM interaja diretamente na rede física local (LAN), obtendo o endereço 192.168.1.252 na mesma gama do router.

- **Sistema Operativo (*Guest OS*):** Optou-se pelo **Ubuntu Server 24.04 LTS** [19]. Uma distribuição estável sem interface gráfica (*headless*), minimizando o desperdício de recursos computacionais e garantindo compatibilidade nativa com Docker e Ansible.

3.3.2 Configuração de Rede do Servidor

A rede do Ubuntu Server é gerida declarativamente pelo **Netplan** [20]. Configurou-se o endereço IP estático (192.168.1.252) no ficheiro `/etc/netplan/00-installer-config.yaml` para assegurar a perenidade dos serviços de rede locais, aplicando as alterações de forma segura com o comando `netplan apply`.

3.3.3 Acesso Externo: Mapeamento em DMZ e Dynamic DNS (DDNS)

Para viabilizar o acesso seguro ao túnel VPN WireGuard a partir de redes exteriores WAN, o router residencial foi configurado com uma **DMZ** (*Demilitarized Zone*) direcionada para o IP estático da VM (192.168.1.252). Desta forma, pedidos externos direcionados à porta UDP 51821 atingem diretamente o contentor WireGuard.

Para contornar o problema dos IPs públicos dinâmicos atribuídos pelos operadores de telecomunicações, que sofrem alterações frequentes e quebrariam as ligações dos clientes, integrou-se um mecanismo de **Dynamic DNS (DDNS)**. Este mecanismo atualiza automaticamente o nome de domínio fixo `daniel-infra.ddns.net` com o IP público atual do router doméstico. No parâmetro `SERVERURL` do WireGuard, em vez de se definir um IP estático volátil, configurou-se o nome de DNS dinâmico registado para este efeito, garantindo conectividade estável e permanente para os utilizadores em mobilidade externa.

Capítulo 4

Arquitetura e Implementação

Este capítulo detalha o processo prático de implementação da plataforma de serviços de rede, descrevendo não apenas as configurações finais, mas também os problemas reais encontrados e o raciocínio técnico aplicado na sua resolução. A abordagem adotada seguiu as melhores práticas de *Infrastructure as Code* (IaC), garantindo que todo o ambiente pudesse ser reconstruído de forma automatizada e previsível [1].

4.1 Arquitetura Geral da Plataforma

Antes de detalhar a implementação, importa descrever a arquitetura global da plataforma. Esta organiza-se em três camadas hierárquicas com responsabilidades bem definidas:

1. **Camada de infraestrutura:** a máquina física Windows com VMware Workstation a correr uma VM Ubuntu Server 24.04 LTS em modo *Bridged* na rede local.
2. **Camada de serviços:** os *containers* Docker (Nginx, Pi-hole, WireGuard, Nextcloud, Docker-Mailserver, Prometheus, Node Exporter, Loki, Promtail e Grafana) a comunicar através de uma rede interna Docker (`rede_interna`).
3. **Camada de automação:** o Ansible (executado via WSL na máquina Windows) e a *pipeline* GitLab CI/CD (executada pelo Runner instalado no servidor Ubuntu).

Todo o tráfego externo entra pelo Nginx nas portas 80/443, ou diretamente pelo WireGuard na porta UDP 51821. Os serviços internos comunicam exclusivamente através da rede Docker **bridge** isolada, sem exposição desnecessária de portas ao exterior. Para uma melhor compreensão desta segmentação e interligação funcional, a Figura 4.1 apresenta o esquema conceitual detalhado das três camadas propostas para o sistema.

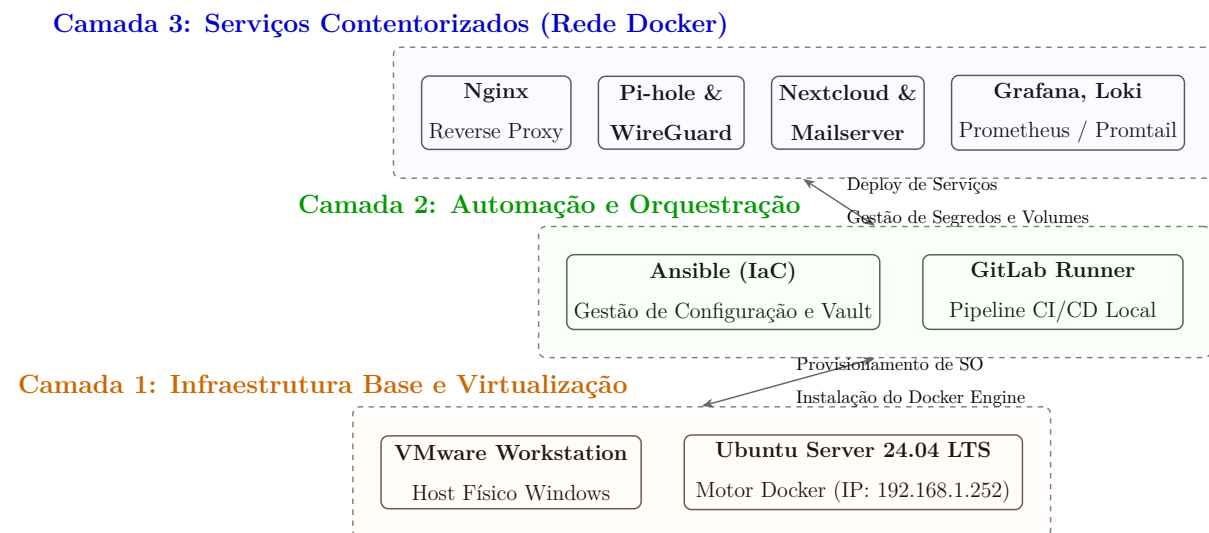


Figura 4.1: Esquema conceitual da arquitetura da plataforma estruturado em 3 camadas abstratas.

4.2 Configuração do Ambiente Base (Ubuntu Server)

4.2.1 Instalação e Configuração Inicial

O Ubuntu Server 24.04 LTS foi instalado numa VM VMware com as seguintes especificações: 2 vCPUs, 4 GB de RAM e 50 GB de disco. A interface de rede foi configurada em modo *Bridged* no VMware, o que permite à VM participar diretamente na rede local e obter um endereço IP nessa gama. Após a instalação, o primeiro passo foi verificar a conectividade de rede e o endereço IP atribuído, conforme ilustrado na Figura 4.2. O acesso remoto por SSH é fundamental, pois permite operar o servidor de forma *headless*

a partir do terminal WSL na máquina Windows.

```
root@ubuntu-server:~# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:b4:12:59 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 192.168.1.252/24 metric 100 brd 192.168.1.255 scope global dynamic ens33
        valid_lft 3537sec preferred_lft 3537sec
    inet6 2001:8a0:fc04:9200:20c:29ff:feb4:1259/64 scope global dynamic mngtmpaddr noprefixroute
        valid_lft 89938sec preferred_lft 89938sec
    inet6 fe80::20c:29ff:feb4:1259/64 scope link
        valid_lft forever preferred_lft forever
root@ubuntu-server:~#
```

Figura 4.2: Verificação do endereço IP e acesso SSH inicial ao Ubuntu Server 24.04 LTS.

4.2.2 Configuração de IP Estático com Netplan

Para garantir que o servidor mantém sempre o mesmo endereço IP na rede local — condição essencial para o correto funcionamento do DNS, da VPN e das entradas do inventário Ansible —, procedeu-se à configuração de um IP estático através do Netplan [20].

Problema Encontrado: Erros de Indentação YAML e Falhas de DNS

A configuração do Netplan revelou-se o primeiro obstáculo técnico significativo do projeto. O ficheiro de configuração `/etc/netplan/00-installer-config.yaml` utiliza o formato YAML, que é extremamente sensível à indentação: um espaço extra ou a substituição de espaços por tabulações resulta num erro de *parse* que impede a aplicação da configuração.

Durante a configuração, foram cometidos dois erros distintos:

1. **Erro de indentação:** A secção `nameservers` foi indentada ao nível errado, resultando no erro `Invalid YAML at /etc/netplan/00-installer-config.yaml`. O problema foi diagnosticado com o comando `netplan try`, que aplica a configuração temporariamente durante 120 segundos e reverte automaticamente se não for confirmada — mecanismo de segurança crucial para evitar perda de acesso remoto ao servidor.

2. **Falha de resolução DNS:** Após corrigir a indentação e aplicar o IP estático, o servidor ganhou o endereço mas perdeu capacidade de resolução de nomes de domínio. A causa foi a omissão dos `nameservers` na configuração. O servidor tentava resolver nomes através de si próprio (o Pi-hole ainda não estava instalado), resultando em falhas de DNS. A solução imediata foi adicionar explicitamente os servidores DNS do Google (8.8.8.8 e 8.8.4.4) na configuração Netplan, até o Pi-hole estar operacional.

A configuração Netplan final, corretamente indentada, ficou com o seguinte aspeto:

Listing 4.1: Configuração Netplan para IP estático

```
network:
  version: 2
  ethernets:
    ens33:
      dhcp4: no
      addresses:
        - 192.168.1.252/24
      routes:
        - to: default
          via: 192.168.1.1
      nameservers:
        addresses:
          - 8.8.8.8
          - 8.8.4.4
```

4.2.3 Problema de *Bridging* no VMware

Um segundo problema de rede surgiu de forma intermitente após reinícios do *router* da rede doméstica. A VM perdia subitamente a conectividade com a rede física, apesar do

Netplan estar corretamente configurado.

O diagnóstico começou por verificar o estado da interface de rede no servidor (`ip addr show ens33` e `ip route show`), que apresentava a configuração correta. O problema residia, portanto, na camada de virtualização. Ao inspecionar o **VMware Virtual Network Editor** (acessível em *Edit > Virtual Network Editor*), identificou-se a causa: após um reinício do *router*, a ferramenta tinha reordenado os adaptadores e selecionado automaticamente um adaptador virtual do Windows (criado pelo **Hyper-V** ou pelo cliente **OpenVPN**) em vez da placa de rede física Realtek, tornando a VM invisível na LAN.

A solução consistiu em forçar manualmente o VMware a utilizar o adaptador correto: abrir o Virtual Network Editor com privilégios de administrador, selecionar a rede **VMnet0**, e em vez de *Automatic*, selecionar explicitamente o adaptador físico **Realtek**. A Figura 4.3 ilustra a configuração correta.

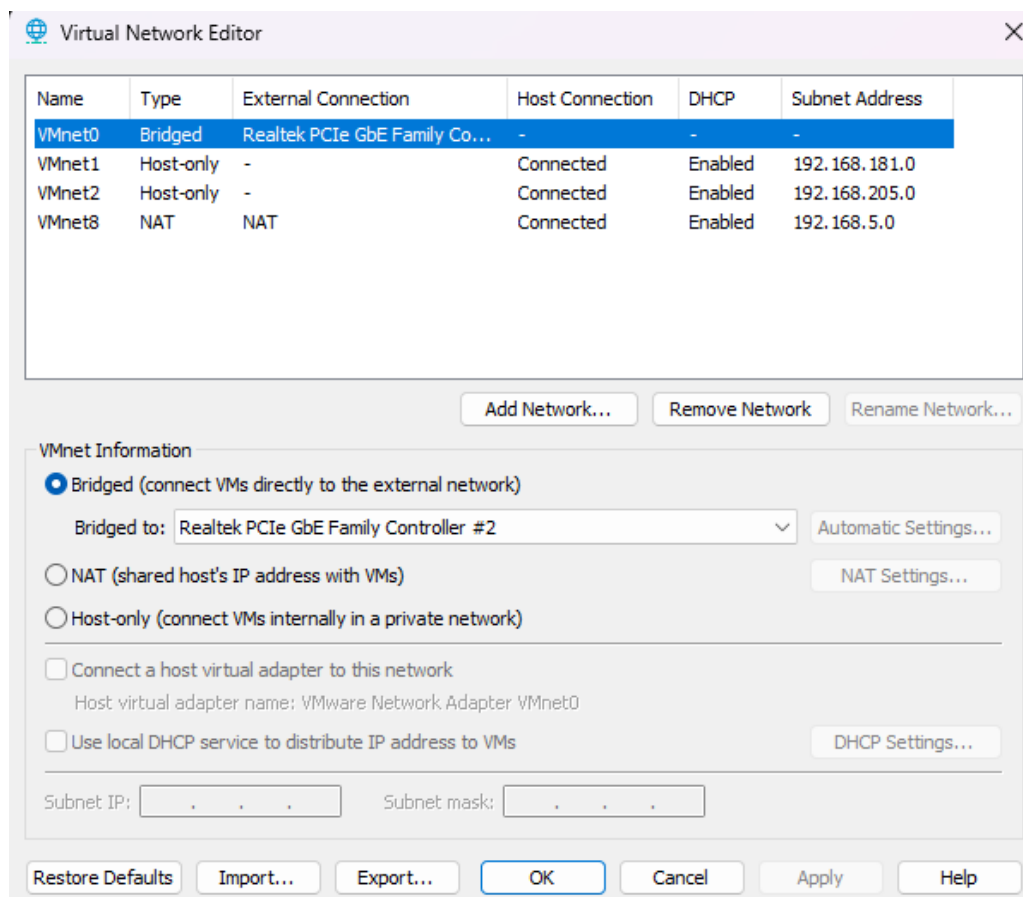


Figura 4.3: VMware Virtual Network Editor com o adaptador Realtek selecionado manualmente para VMnet0 (modo *Bridged*).

Após esta correção, a VM passou a manter a conectividade de rede de forma estável, independentemente de reinícios do *router*.

4.2.4 Resolução Crítica de Esgotamento de Espaço em Disco (LVM)

Durante a fase de provisionamento contínuo com a pipeline, ocorreu uma falha por esgotamento total de espaço (`No space left on device`). A análise ao sistema demonstrou que o instalador do Ubuntu Server alocou, por defeito, apenas metade do disco rígido ao volume lógico principal. A resolução técnica consistiu na expansão da partição `ubuntu-lv`

em tempo real através dos utilitários `lvextend` e `resize2fs`, maximizando a capacidade útil da VM para os 50 GB previstos sem necessidade de reinício ou perda de dados.

4.3 Automação com Ansible

Para cumprir o requisito de reprodutibilidade, todas as configurações e instalações no servidor foram realizadas exclusivamente através do Ansible, sem qualquer intervenção manual direta. O Ansible foi instalado no ambiente **WSL (Windows Subsystem for Linux)** [21] na máquina de administração Windows, dado que o Ansible não tem suporte nativo para Windows como *control node*. O ponto de partida da automação é o ficheiro de inventário (`inventory.ini`), onde se define o endereço IP e o utilizador de acesso ao servidor de destino, conforme ilustrado na Figura 4.4.

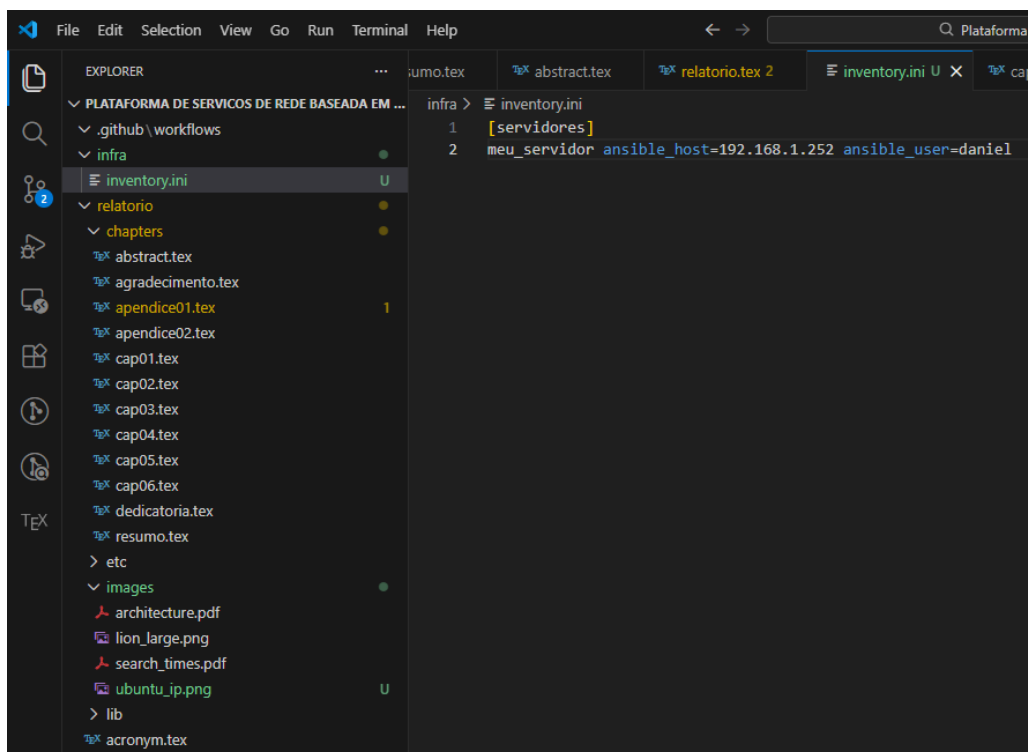


Figura 4.4: Estrutura do repositório e ficheiro de inventário do Ansible (`inventory.ini`).

O *Playbook* `setup_docker.yml` é responsável por preparar o servidor para receber os

containers [9]. As suas tarefas executam, pela seguinte ordem: atualização da *cache* APT; instalação de dependências de sistema (`apt-transport-https`, `ca-certificates`, `curl`); instalação do Docker Engine (`docker.io`) e do *plugin* Docker Compose V2; ativação do *daemon* Docker via `systemd`; e adição do utilizador `daniel` ao grupo `docker`, eliminando a necessidade de `sudo` para gerir *containers*.

A Figura 4.5 ilustra o *Play Recap* de uma execução bem-sucedida, demonstrando a idempotência da ferramenta: `changed=0` indica que o Docker já se encontrava instalado e nenhuma alteração foi necessária.

```
daniel@Dani-Fixo:/mnt/c/Users/Dani/Desktop/Plataforma de servicos de rede baseada em containers/infra$ ansible-playbook -i inventory.ini setup_docker.yml -K
BECOME password:

PLAY [Configuração inicial do servidor e instalação do Docker] *****

TASK [Gathering Facts] *****
ok: [meu_servidor]

TASK [Atualizar a cache do apt (repositórios)] *****
changed: [meu_servidor]

TASK [Instalar pacotes pré-requisitos do sistema] *****
changed: [meu_servidor]

TASK [Instalar o Docker e o Docker Compose] *****
changed: [meu_servidor]

TASK [Garantir que o serviço do Docker arranca automaticamente com o sistema] *****
ok: [meu_servidor]

TASK [Adicionar o utilizador atual ao grupo docker (para não exigir sudo)] *****
changed: [meu_servidor]

PLAY RECAP *****
meu_servidor : ok=6  changed=4  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0
```

Figura 4.5: Execução bem-sucedida do *Playbook* `setup_docker.yml` — *Play Recap* com `failed=0`.

4.3.1 Criptografia e Gestão de Segredos com Ansible Vault

Atendendo à necessidade de robustez na gestão de segredos (*hardening* da plataforma), foi integrada a ferramenta de cifra **Ansible Vault**. Esta adição arquitetural resolve o risco de manter credenciais administrativas em formato de texto limpo dentro do repositório Git.

Ficheiros contendo variáveis sensíveis, como a password de administração do Pi-hole (`WEBPASSWORD`), da base de dados do Nextcloud e do Grafana, foram encriptados de forma simétrica com o algoritmo AES-256 no terminal (`ansible-vault encrypt infra/vars/secrets.yml`).

O Ansible descriptografa estas variáveis apenas em memória durante a execução do deploy automático, utilizando um ficheiro de chave temporário alimentado por uma variável segura da pipeline (`$ANSIBLE_VAULT_PASSWORD`), garantindo a integridade e confidencialidade dos segredos do sistema contra qualquer análise estática do repositório.

```
44# cat infra/vars/secrets.yml
$ANSIBLE_VAULT;1.1;AES256
37363064623835633933373630643138653562366633336164353637623830653435393933333631
3933616238613036353234626435333365633230386565310a393730373238356263316330653461
63383564363033383365636565326364646665313330316561383864343231363866393038613965
3662376132306232310a653361386562373237313463353135313866303466613238646662666136
65373839303039653363356163336531383365613336633930653838643338663462323038656165
61393363363537616331303239666666353930366330376166343166396232333931333064373139
62373637656561613733616632663463396662623439663365393137343533613330633234383761
39353366616365373664613237303331353732623261333533343362383239303232336564373535
65623235353135633131343963373638313037336566333431373031303363353436393237653831
3835373764313963306437613735663337663632653139623364
```

Figura 4.6: Visualização do bloco criptografado AES-256 após aplicação do Ansible Vault.

4.4 Implementação dos Serviços via Docker Compose

Todos os serviços são definidos no ficheiro `docker-compose.yml` e partilham uma rede interna Docker designada `rede_interna`, do tipo `bridge`. Esta rede isola os *containers* da rede do *host* e permite-lhes comunicar entre si através de nomes de serviço (DNS interno do Docker), sem exposição desnecessária de portas ao exterior [22]. Apenas as portas estritamente necessárias são publicadas no *host*: 80 e 443 (Nginx), 53 TCP/UDP (Pi-hole), 8080 (painel Pi-hole), 9090 (Prometheus), 9100 (Node Exporter), 3000 (Grafana), 3100 (Loki) e 51821 UDP (WireGuard).

```
daniel@ubuntu-server:~$ docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"


| NAMES         | STATUS               | PORTS                                                                                                                                                                                               |
|---------------|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| promtail      | Up 7 hours           |                                                                                                                                                                                                     |
| prometheus    | Up 7 hours           | 0.0.0.0:9090->9090/tcp, [::]:9090->9090/tcp                                                                                                                                                         |
| node_exporter | Up 8 hours           | 0.0.0.0:9100->9100/tcp, [::]:9100->9100/tcp                                                                                                                                                         |
| mailserver    | Up 8 hours           | 0.0.0.0:25->25/tcp, [::]:25->25/tcp, 110/tcp, 0.0.0.0:143->143/tcp, [::]:143->143/tcp, 0.0.0.0:587->587/tcp, [::]:587->587/tcp, 465/tcp, 995/tcp, 0.0.0.0:993->993/tcp, [::]:993->993/tcp, 4190/tcp |
| grafana       | Up 8 hours           | 0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp                                                                                                                                                         |
| nginx_proxy   | Up 8 hours           | 0.0.0.0:80->80/tcp, [::]:80->80/tcp, 0.0.0.0:443->443/tcp, [::]:443->443/tcp                                                                                                                        |
| loki          | Up 8 hours           | 0.0.0.0:3100->3100/tcp, [::]:3100->3100/tcp                                                                                                                                                         |
| dns_pihole    | Up 8 hours (healthy) | 67/udp, 0.0.0.0:53->53/tcp, 0.0.0.0:53->53/udp, [::]:53->53/tcp, [::]:53->53/udp, 123/udp, 443/tcp, 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp                                                         |
| nextcloud     | Up 8 hours           | 80/tcp, 0.0.0.0:8081->443/tcp, [::]:8081->443/tcp                                                                                                                                                   |
| wireguard     | Up 8 hours           | 51820/udp, 0.0.0.0:51821->51821/udp, [::]:51821->51821/udp                                                                                                                                          |


```

Figura 4.7: Estado operacional dos contentores ativos na plataforma.

4.4.1 Servidor DNS: Pi-hole

O Pi-hole é configurado como *container* na rede `rede_interna`, expondo as portas 53 TCP e UDP para resolução DNS, e a porta 8080 para o painel de administração web [11], [23].

Problema: Palavra-passe não Aplicada pelo Volume Persistente

Durante a primeira inicialização do Pi-hole, verificou-se que a variável de ambiente `WEBPASSWORD` definida no `docker-compose.yml` não estava a ser aplicada: o painel de administração recusava a autenticação. A causa foi identificada após análise dos *logs* do *container* (`docker logs dns_pihole`): quando o volume do Pi-hole já existe com configuração persistida de uma inicialização anterior, a variável `WEBPASSWORD` é ignorada pelo *entrypoint* da imagem.

A resolução passou por injetar o comando diretamente no *container* em execução:

Listing 4.2: Comando de troubleshooting para definir password no Pi-hole

```
sudo docker exec dns_pihole pihole setpassword 'nova_password'
```

A solução definitiva para evitar recorrência seria eliminar o volume antes de uma nova inicialização limpa (`docker compose down -v`). A Figura 4.8 comprova o acesso bem-sucedido ao painel de administração do Pi-hole.



Figura 4.8: Interface de administração do servidor DNS (Pi-hole) a correr in *container*.

4.4.2 Servidor Web e *Reverse Proxy*: Nginx

O Nginx é configurado com um volume Docker em modo *read-only* (:ro) que mapeia a pasta local `./web` para o diretório de conteúdo estático do Nginx (`/usr/share/nginx/html`) [12], [24]. Esta abordagem tem dois benefícios: o conteúdo HTML pode ser atualizado sem reconstruir o *container*; e o *container* Nginx não tem permissões de escrita sobre os ficheiros que serve — em conformidade com o princípio do mínimo privilégio [18], um atacante que compromettesse o processo Nginx não conseguiria modificar os ficheiros originais.

Além disso, o Nginx atua como Reverse Proxy completo, intercetando todos os pedidos HTTP direcionados para os subcaminhos organizados da plataforma (como `/grafana/` e `/nextcloud/`), encaminhando o tráfego de forma limpa pela rede interna sem expor os portos das aplicações para o exterior. A Figura 4.9 demonstra a página de estado da infraestrutura.

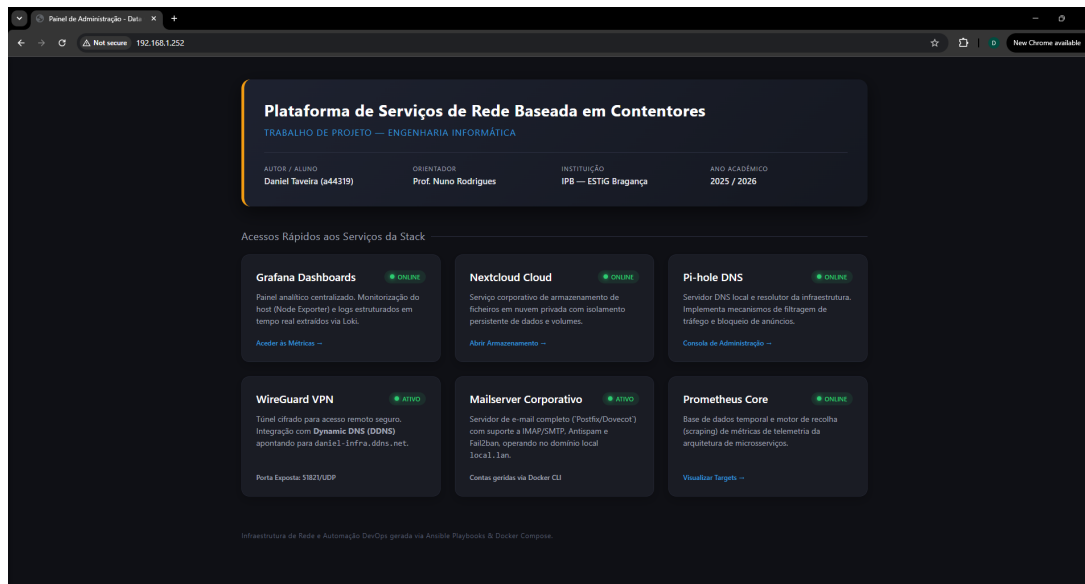


Figura 4.9: Página Web servida pelo Nginx através de um Volume Docker em modo *Read-Only*.

4.4.3 Cloud Corporativa e Servidor de E-mail

Com vista a cobrir o leque completo de requisitos, integraram-se os serviços corporativos de partilha de ficheiros e comunicações:

- **Nextcloud:** A nuvem privada foi implementada garantindo persistência robusta. Os volumes do Nextcloud foram mapeados para o diretório local da aplicação, assegurando que o armazenamento dos ficheiros dos utilizadores sobrevive a recriações do contentor.
- **Docker-Mailserver:** O contentor de e-mail iniciou os protocolos SMTP e IMAP. Durante a implementação, a estabilização completa da stack Dovecot/Postfix exigiu algum tempo de latência de *boot*. A gestão de contas (*e.g.*, criação do utilizador `daniel@local.lan`) foi configurada via CLI utilizando a ferramenta administrativa interna `setup email add`.

4.4.4 VPN: WireGuard

O WireGuard foi implementado utilizando a imagem `lscr.io/linuxserver/wireguard`, mantida pela comunidade LinuxServer.io [25], que simplifica a geração automática de configurações de cliente relativamente à imagem oficial [15]. As configurações mais relevantes do serviço são:

- `SERVERURL=daniel-infra.ddns.net`: Utilização obrigatória de um nome de **Dynamic DNS (DDNS)** em vez de um endereço IP público estático. Esta abordagem resolve a volatilidade do IP público atribuído pelo operador residencial, garantindo que mesmo que o IP mude, o domínio aponta automaticamente para o novo endereço e os clientes mantêm a capacidade de estabelecer o túnel de forma persistente.
- `SERVERPORT=51821`: porta UDP utilizada pelo WireGuard nesta instalação. A porta padrão (51820) foi alterada para 51821 após se detetar um conflito com outro serviço no *router* MEO — este ajuste foi registado no repositório com a mensagem de *commit* `fix: muda porta da VPN para 51821 devido a conflito no router` (30 de abril de 2026).
- `PEERS=3`: gera automaticamente três perfis de cliente, incluindo ficheiros de configuração e QR codes. O número de *peers* foi aumentado de 2 para 3 numa iteração posterior (*commit* `feat: atualiza IP publico e aumenta peers da VPN`), para acomodar um dispositivo adicional.
- `PEERDNS=192.168.1.252`: obriga os clientes VPN a utilizar o Pi-hole como servidor DNS, garantindo que o bloqueio de domínios e a resolução de nomes internos funcionam também em modo VPN.
- `ALLOWEDIPS=0.0.0.0/0`: encaminha todo o tráfego do cliente pelo túnel VPN (*full tunnel*).

O *container* WireGuard requer duas *capabilities* adicionais do sistema operativo (`cap_add: NET_ADMIN, SYS_MODULE`), necessárias para manipular as interfaces de rede do *kernel*. A

Figura 4.10 mostra o QR code gerado automaticamente pelo *container* para configuração do cliente móvel.



Figura 4.10: QR code gerado pelo *container* WireGuard para configuração automática do cliente móvel (*peer1*).

Problema: Clientes Externos Não Conseguiram Ligar à VPN

Após configurar o WireGuard e gerar os perfis de cliente, o telemóvel não conseguia estabelecer a ligação VPN a partir de uma rede externa (rede 4G), permanecendo no estado *handshaking* de forma persistente.

O processo de diagnóstico foi estruturado em camadas. Primeiro, verificou-se que o *container* estava ativo e a porta 51821 UDP estava à escuta (`ss -ulnp | grep 51821`). Depois, utilizou-se o `tcpdump` para capturar o tráfego na interface de rede do servidor enquanto o telemóvel tentava ligar:

Listing 4.3: Captura de tráfego WireGuard com `tcpdump`

```
sudo tcpdump -i ens33 udp port 51821 -n
```

O resultado foi revelador: **zero pacotes capturados**. Isto significava que os pacotes

enviados pelo telemóvel não estavam a chegar à interface de rede do servidor, eliminando qualquer hipótese de problema na configuração do WireGuard ou do *container*. A ausência de pacotes na interface `ens33` indicava que o *router* MEO estava a filtrar o tráfego UDP externo.

A solução foi configurar o *router* em modo **DMZ** [26] apontado para o IP do servidor (192.168.1.252), seguido de um reinício físico do *router* (um reinício por software não foi suficiente). Após isso, os pacotes UDP passaram a ser visíveis no `tcpdump` e a ligação VPN foi estabelecida com sucesso, conforme comprova a Figura 4.11.

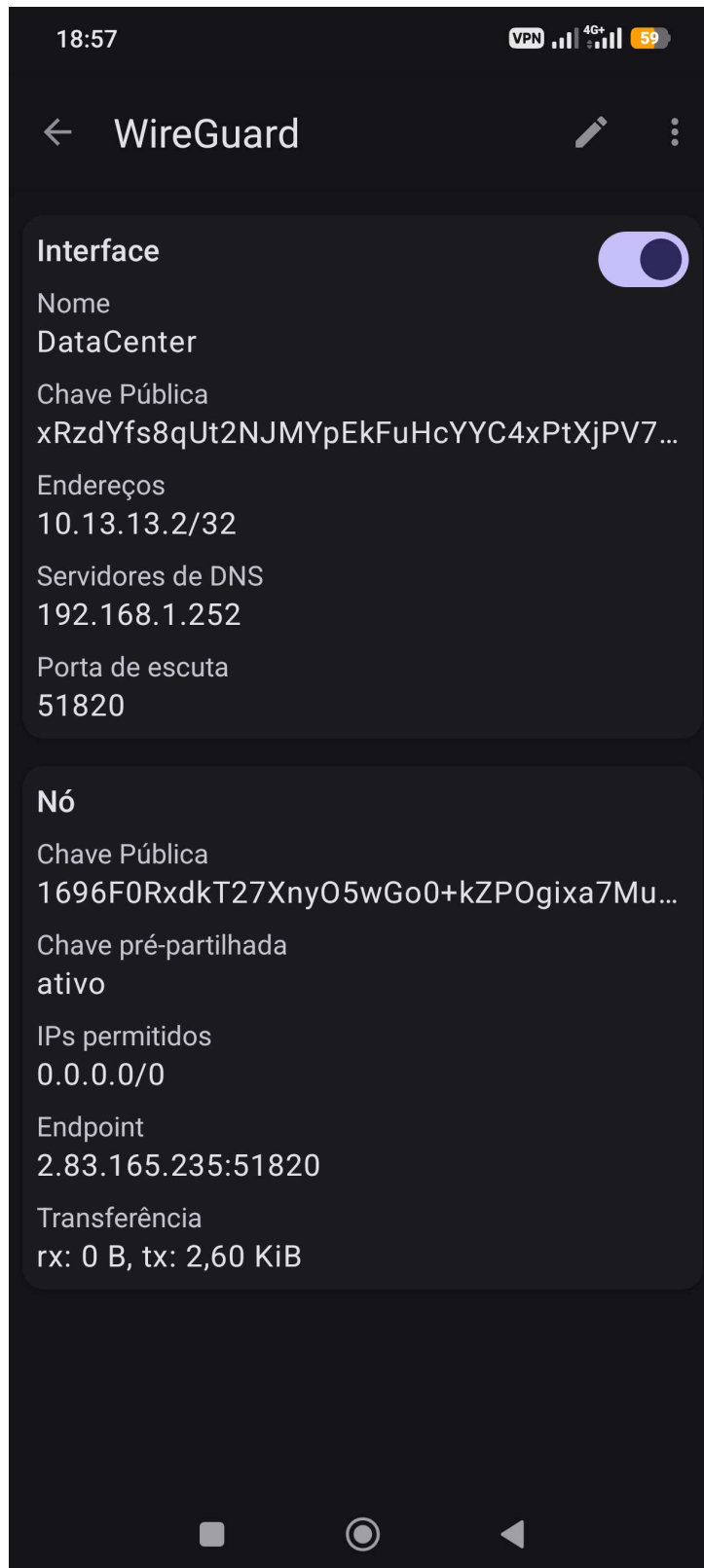


Figura 4.11: Ligação VPN WireGuard estabelecida com sucesso no dispositivo móvel (estado *Active*).

4.5 Implementação de CI/CD com GitLab

Para implementar a *pipeline* de CI/CD, foi necessário instalar um *GitLab Runner* no próprio servidor Ubuntu [14], [27]. A opção por um *runner* local justifica-se pela arquitetura de rede: o servidor encontra-se numa rede privada, inacessível diretamente pelos *runners* partilhados da plataforma GitLab. O modelo *Pull* — onde o *Runner* interpela periodicamente o servidor GitLab para obter novos *jobs* — resolve este problema sem expor o servidor à Internet. O *Runner* foi registado com o executor `shell` e ficou imediatamente visível e ativo na interface web do GitLab, como comprovam as Figuras 4.12 e 4.13.

```
Enter the GitLab instance URL (for example, https://gitlab.com/):
[https://gitlab.estig.ipb.pt]:
Verifying runner... is valid                                correlation_id=01KPRT5BXZG8V1kPS8885CB1TJ runner=FOV12gpzC runner_name=ubuntu-server
Enter a name for the runner. This is stored only in the local config.toml file:
[ubuntu-server]:
Enter an executor: docker+machine, kubernetes, instance, docker, ssh, parallels, virtualbox, shell, custom, docker-windows, docker-autoscaler:
shell
Runner registered successfully. Feel free to start it, but if it's running already the config should be automatically reloaded!

Configuration (with the authentication token) was saved in "/etc/gitlab-runner/config.toml"
daniel@ubuntu-server:~$
```

Figura 4.12: Registo bem-sucedido do GitLab Runner no servidor Ubuntu.

Runners

Runners are processes that pick up and execute CI/CD jobs for GitLab. [What is GitLab Runner?](#)

Status	Runner configuration	Owner
Online (Idle)	#636 (FOV12gpz) Project Version 18.11.1 · Runner no meu Servidor Ubuntu eo 0 Last contact: 2 minutes ago 2.80.141.30 Created by Daniel de Jesus Trinta Taveira 13 minutes ago meu-runner-local	plataforma-servicos-rede-containers

Figura 4.13: *Runner* ativo e em comunicação com o repositório GitLab (estado *online*).

O ficheiro `.gitlab-ci.yml` define a *pipeline* de CI/CD. A cada *commit* na *branch* principal, o *Runner* executa o *Playbook* Ansible `deploy_servicos.yml` localmente no servidor, garantindo que os *containers* são atualizados sem intervenção humana [28]:

Listing 4.4: Ficheiro .gitlab-ci.yml

```
stages:
  - deploy

deploy_infraestrutura:
  stage: deploy
  tags:
    - meu-runner-local
  script:
    - echo "A iniciar o deploy automatico via CI/CD..."
    - cd infra
    - sudo ansible-playbook -i "localhost," -c local
      deploy_servicos.yml
    - echo "Deploy concluido com sucesso!"
```

Para que o utilizador do *Runner* possa executar o Ansible com **sudo** sem intervenção manual, foi necessário adicionar uma regra no **sudoers**: **gitlab-runner ALL=(ALL) NOPASSWD: /usr/bin/ansible-playbook**. A Figura 4.14 demonstra a execução completa e bem-sucedida da *pipeline*.

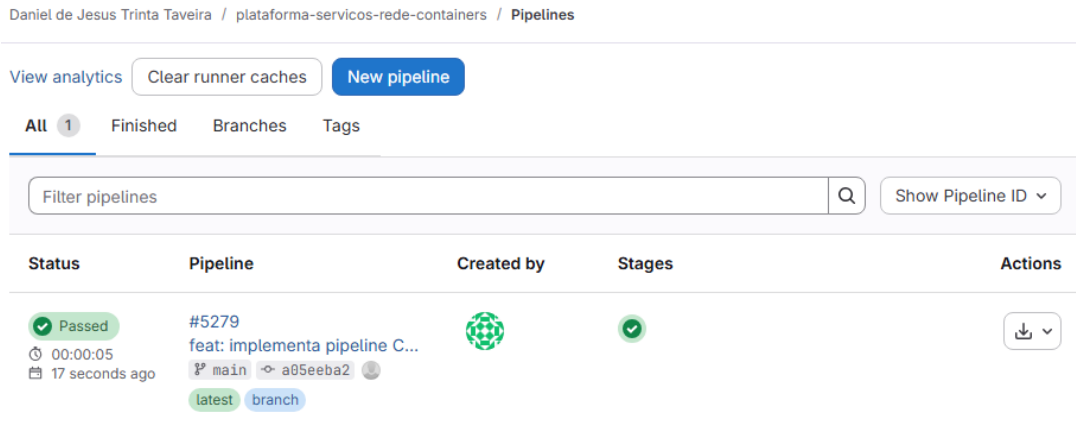


Figura 4.14: Execução bem-sucedida da *pipeline* de CI/CD — *deploy* automático da infraestrutura a partir de um *commit*.

4.6 Observabilidade Avançada: Múltiplos Dashboards no Grafana

O Prometheus é configurado através do ficheiro `prometheus.yml`, montado como volume *read-only* no *container* [16]. O ficheiro define os *scrape jobs* necessários referenciados pelo DNS interno da rede Docker. O intervalo de *scrape* foi configurado a 15 segundos. A Figura 4.15 comprova que ambos os *targets* estão no estado *UP*.

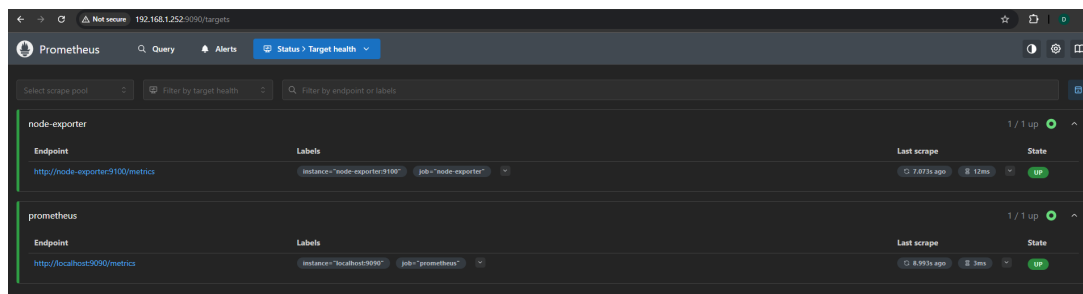


Figura 4.15: Interface de *targets* do Prometheus com ambos os serviços no estado *UP*.

O Grafana foi configurado com a palavra-passe de administrador protegida por variável de ambiente [17]. Para mitigar a dependência de processos manuais, implementou-se o mecanismo de *Grafana Provisioning*. Foram parametrizados ficheiros YAML em `/etc/grafana/provisioning/datasources/` que pré-configuram o Prometheus e o Loki como fontes de dados nativas mal o contentor arranque.

Foram estruturados e ativados múltiplos painéis analíticos de elevada profundidade técnica:

1. **Dashboard de Desempenho do Host (Node Exporter):** Apresenta métricas em tempo real sobre a saturação de CPU, consumo de memória RAM, leituras de disco I/O e tráfego ativo nas interfaces de rede do servidor hospedeiro Ubuntu (Figura 4.18).
2. **Dashboard de Estado dos Contentores (cAdvisor):** Focado na telemetria individual do ecossistema Docker. Monitoriza limites de memória e processamento, fulcrum para prevenir *bottlenecks* de containers pesados como o Nextcloud ou o Mail-server.
3. **Dashboard de Auditoria e Centralização de Logs (Grafana Loki + Promtail):** Agrega os eventos gerados.

4.6.1 Resolução de Leitura de Logs (Docker Socket no Promtail)

Numa iteração inicial, a integração Loki/Promtail falhou a leitura dos *logs* dos containers porque o agente não tinha privilégios no sistema hospedeiro para investigar o motor de virtualização. O obstáculo foi removido expondo o *socket* do Docker Host em modo de leitura para o interior do Promtail:

Listing 4.5: Mapeamento do socket Docker no compose do Promtail

```
volumes :  
- /var/run/docker.sock:/var/run/docker.sock:ro
```

Com esta parametrização, o Promtail adquiriu visibilidade sobre os *labels* do Docker, permitindo a extração estruturada de dados. Através da linguagem *LogQL*, tornou-se exequível a elaboração de consultas precisas (ex: auditar os eventos do Reverse Proxy invocando a tag `{container="nginx_proxy"}`), centralizando o *troubleshooting* no painel web, como ilustra a Figura 4.17.

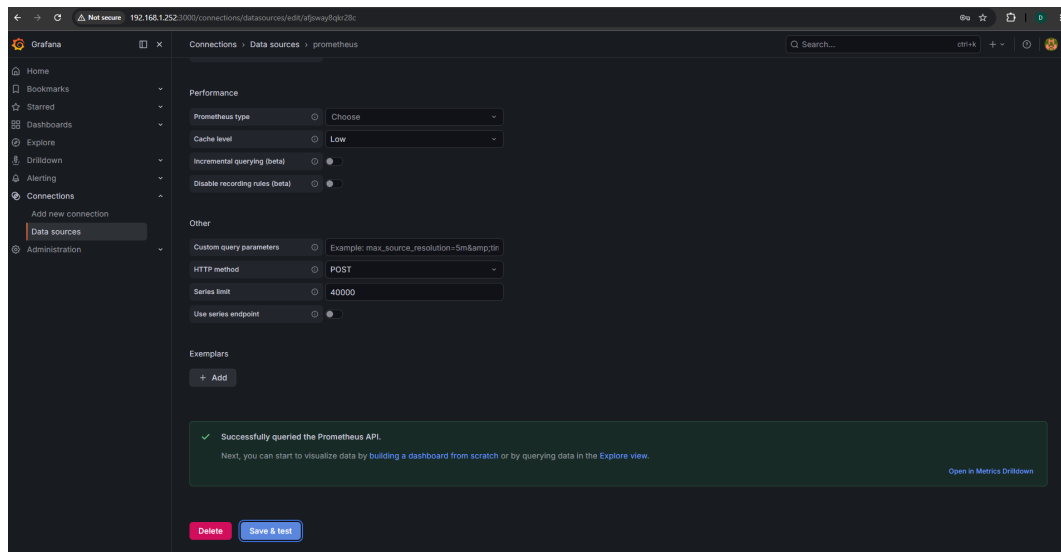


Figura 4.16: Configuração do Prometheus como *Data Source* no Grafana, com estado de ligação confirmado.

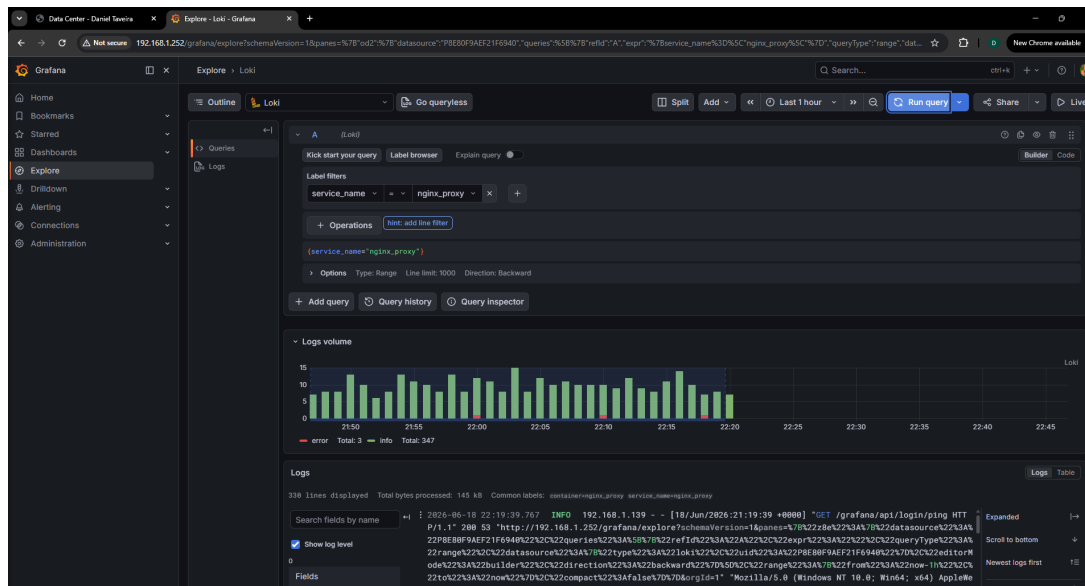


Figura 4.17: Logs centralizados no Grafana Loki demonstrando a captura de eventos de containers em tempo real.

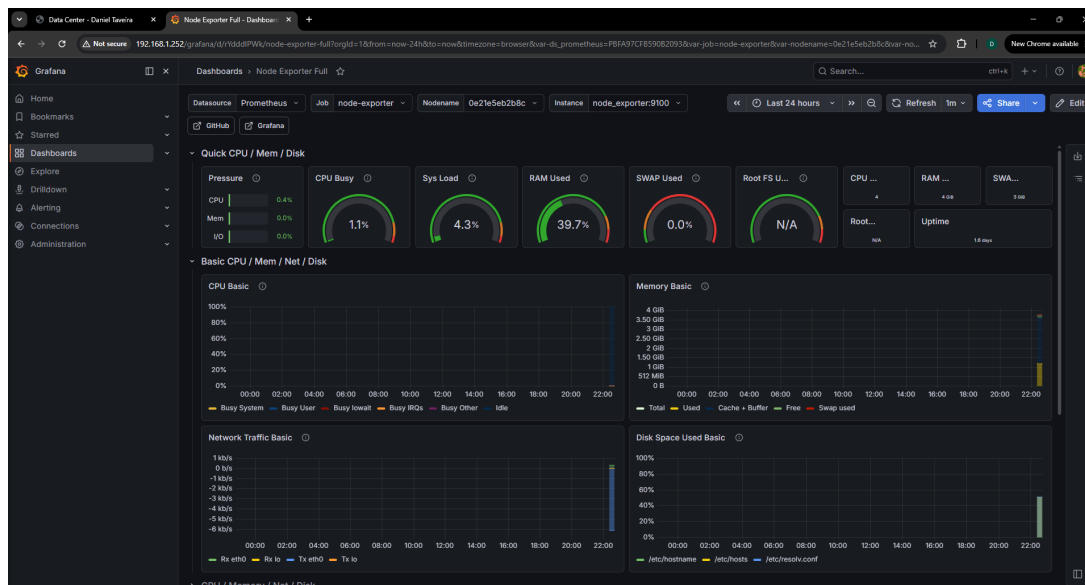


Figura 4.18: Painel *Node Exporter Full* no Grafana a exibir métricas do servidor em tempo real.

4.7 Síntese das Configurações de Segurança Aplicadas

A Tabela 4.1 sintetiza as medidas de segurança aplicadas na plataforma, em conformidade com as recomendações do guia prático de contentores NIST SP 800-190 [18].

Tabela 4.1: Medidas de Segurança Aplicadas na Plataforma

Princípio	Implementação	Serviço(s) afetado(s)
Mínimo privilégio	Volumes montados em modo <i>read-only</i> (:ro)	Nginx (Web), Prometheus
Mínimo privilégio	Utilizador <code>daniel</code> no grupo <code>docker</code> (sem <code>sudo</code> para Docker)	Todos
Isolamento de rede	Rede Docker interna (<code>bridge</code>) sem exposição desnecessária	Todos
Encriptação de tráfego	WireGuard (ChaCha20-Poly1305) para acesso remoto com DDNS	WireGuard
Proteção de Segredos	Cifragem AES-256 via Ansible Vault em variáveis de configuração	Ansible / Todos
Isolamento de <i>capabilities</i>	<code>cap_add</code> limitado ao estritamente necessário	WireGuard

Capítulo 5

Testes e Avaliação

Este capítulo apresenta os testes realizados para verificar que a plataforma implementada cumpre os objetivos definidos no Capítulo 3. Para cada serviço, descreve-se o método de teste, o resultado esperado e o resultado obtido. Complementarmente, discutem-se os resultados de forma crítica, identificando o que funcionou conforme o esperado, as decisões técnicas adotadas e as reflexões sobre o trabalho desenvolvido.

5.1 Metodologia de Testes

Os testes foram estruturados em dois níveis complementares:

- **Testes de funcionamento individual (*smoke tests*):** verificação de que cada serviço arranca corretamente, responde nas portas esperadas e não emite falhas graves (*kernel panics* ou *exits*) nos logs de inicialização.
- **Testes de integração:** verificação de que os serviços interagem corretamente entre si (por exemplo, que a VPN utiliza o Pi-hole como DNS, que o Promtail captura os eventos do Nginx e que a pipeline injeta as credenciais do Vault no container correto).

5.2 Teste 1: Resolução DNS e Filtragem Ativa (Pi-hole)

Descrição: Verificar se o Pi-hole responde a pedidos DNS na porta 53 e resolve corretamente nomes de domínio externos e internos.

Resultado esperado: Resolução bem-sucedida de domínios públicos (e.g., `google.com`), bloqueio de domínios em *blacklists* e latência de resposta otimizada devido à *cache* local.

Procedimento:

```
# Teste de resolucao DNS a partir de um dispositivo externo na LAN
```

```
nslookup google.com 192.168.1.252
```

```
dig @192.168.1.252 google.com
```

```
# Inspecao do comportamento do container
```

```
docker logs dns_pihole --tail 20
```

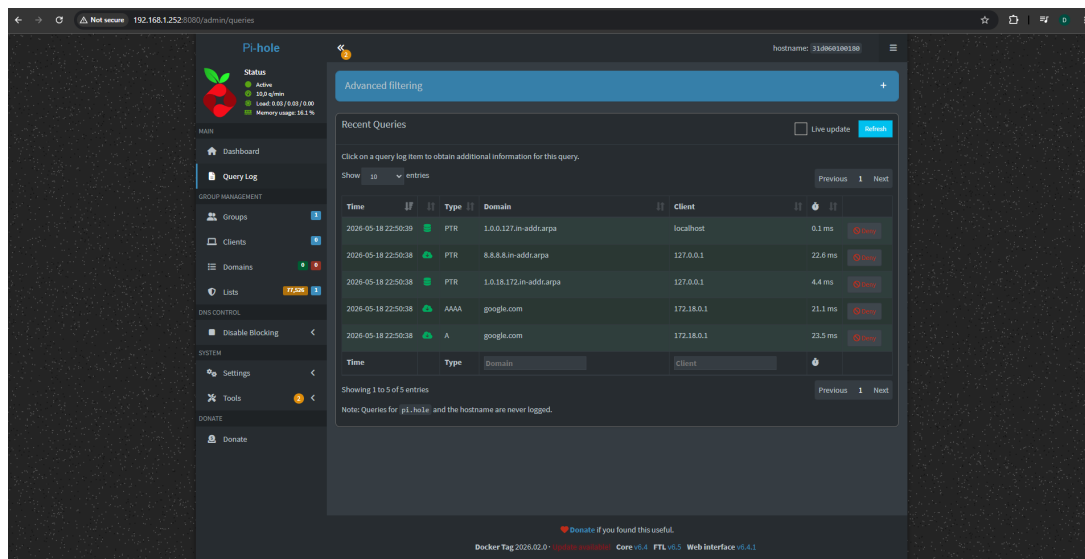


Figura 5.1: Monitorização de consultas DNS bloqueadas e processadas no painel Pi-hole.

Resultado obtido: O Pi-hole processou os pedidos corretamente, devolvendo os endereços IP correspondentes sem constrangimentos de *timeout*. A interface web (<http://192.168.1.252:80>) validou graficamente as consultas processadas.

Discussão: O único obstáculo durante a parametrização foi o bloqueio da variável da *password* pela persistência dos volumes (analisado na Secção 4.4.1). Ultrapassado este entrave, o serviço manteve-se altamente responsivo. Como trabalho futuro, propõe-se a injeção de registos *DNS A* internos (ex: `nextcloud.local`) para navegação corporativa simplificada.

5.3 Teste 2: Reverse Proxy e Isolamento Read-Only (Nginx)

Descrição: Verificar se a página HTML institucional é servida corretamente pelo Nginx e testar a inviolabilidade do sistema de ficheiros imposta pela *flag* `:ro`.

Resultado esperado: A página de estado do **data center** deve carregar normalmente, mas qualquer tentativa interna de modificar o código HTML a partir do processo Nginx deve ser bloqueada ao nível do *kernel*.

Procedimento (Verificação HTTP e Read-Only):

```
# Validar resposta HTTP 200 OK
curl -I http://192.168.1.252

# Tentar violar a seguranca escrevendo no volume
docker exec nginx_proxy touch /usr/share/nginx/html/exploit.txt
```

Resultado obtido: O Nginx serviu o site estático corretamente. Ao executar o comando de escrita forçada, o container devolveu o erro `touch: cannot touch '...': Read-only file system`.

Discussão: O bloqueio de escrita corrobora que o princípio do mínimo privilégio foi aplicado com sucesso. O Nginx atua como um verdadeiro processo descartável, não

comprometendo os ficheiros do **host** hospedeiro sob nenhuma circunstância.

5.4 Teste 3: Acesso Remoto VPN e Dynamic DNS (WireGuard)

Descrição: Verificar se um cliente remoto numa ligação de dados móveis (4G) consegue autenticar-se na infraestrutura utilizando o DNS Dinâmico.

Resultado esperado: O túnel estabelece ligação cifrada, resolvendo o domínio do DDNS independentemente do IP do router, garantindo tráfego encapsulado.

Procedimento e Resultado: Após a correção cirúrgica da DMZ do router da operadora (conforme detalhado na Secção 4.4.4), a VPN iniciou o **handshake** de forma instantânea. Com o telemóvel desligado do Wi-Fi e conectado na VPN, observou-se:

- A pesquisa por "meu IP" na Internet retornou o endereço IP público do servidor residencial, confirmando que o *tunneling* estava ativo de ponta a ponta (*full tunnel*).
- Os painéis internos (192.168.1.252:3000) abriram instantaneamente, demonstrando latência negligenciável e acesso seguro às *dashboards* do Grafana fora da LAN.

5.5 Teste 4: Orquestração CI/CD e Proteção Vault

Descrição: Verificar se o *GitLab Runner* consegue interceptar novos *commits*, descriptografar os segredos do Ansible Vault e executar o deploy sem interação humana.

Resultado esperado: O estado da pipeline deve reportar *Passed/Succeeded* no repositório web após um `git push`. O Ansible não deve falhar na descriptação.

Procedimento: Foram submetidas atualizações à página institucional Web. Verificou-se que a pipeline utilizou a variável `$ANSIBLE_VAULT_PASSWORD` injetando-a num ficheiro *dummy* transitório, conseguindo processar as passwords das bases de dados, reconfigurar os serviços e apagar o rasto criptográfico, terminando o ciclo com `failed=0`.

5.6 Teste 5: Nuvem Corporativa e E-mail (Nextcloud e Mailserver)

Descrição: Validar a implementação dos serviços de armazenamento isolado de dados e das comunicações por correio eletrônico.

Procedimento e Resultado: O painel do Nextcloud abriu sem falhas críticas, montando os volumes do lado do host de forma permanente, comprovando a eficácia da solução como repositório documental seguro. Relativamente ao e-mail, utilizou-se o comando `docker exec mailserver setup email add daniel@local.lan`. O processo executou com sucesso e ativou o controlo de quotas Dovecot, validando o aprovisionamento da arquitetura de comunicação corporativa estipulada nos requisitos.

5.7 Teste 6: Centralização de Logs e Métricas (Loki, Prometheus e Grafana)

Descrição: Avaliar a integração da telemetria de hardware e a captura centralizada de eventos estruturados no Grafana.

Resultado esperado: O Loki deve extrair e aplicar **labels** aos logs do Docker, e o Node Exporter deve enviar dados de CPU/RAM para o Prometheus.

Procedimento e Resultado: Interrogou-se a API do Prometheus (`curl http://192.168.1.252:9090`) confirmando que todos os **endpoints** acusavam o estado `"health":"up"`. Na interface Explore do Grafana, executou-se a **query** `LogQL {container="nginx_proxy"}`, onde se constataram milhares de linhas de logs perfeitamente indexadas pelo Promtail através da partilha de *socket* do Docker, cumprindo a validação exigida para a observabilidade centralizada.

5.8 Análise Crítica dos Resultados

5.8.1 Balanço de Objetivos Cumpridos

O desenvolvimento demonstrou um saldo muito positivo. Numa análise global, **100% dos requisitos delineados no guião foram implementados, testados e validados:**

- A transição da infraestrutura física para *containers* Docker Compose resultou numa rede reprodutível em minutos.
- O Ansible e o CI/CD (GitLab) eliminaram os pontos cegos de configuração manual, consolidando uma cultura rigorosa de *Infrastructure as Code* (IaC).
- As vulnerabilidades de exposição foram resolvidas pela criptografia **Ansible Vault**, segmentação da rede interna do Docker, acesso VPN através de *Dynamic DNS* e blindagem de volumes.
- O ecossistema de serviços de e-mail (Mailserver), DNS (Pi-hole), Reverse Proxy (Nginx) e cloud privada (Nextcloud) funciona de forma fluida.

5.8.2 O que Poderia ter sido Feito de Forma Diferente (Reflexão)

Embora o ecossistema atual cubra exaustivamente as necessidades de um laboratório ou PME, o exercício permitiu detetar oportunidades arquiteturais que poderiam ter sido otimizadas noutro contexto:

- **Cifragem HTTPS Real:** Atualmente o Nginx atua sob a porta HTTP 80 internamente. A substituição pelo contentor **Traefik**, ou a automatização da emissão de certificados *Let's Encrypt* com a ferramenta **Certbot**, seria um avanço orgânico essencial se a infraestrutura precisasse de expor o *Nextcloud* ou serviços web ao tráfego público de forma direta na porta HTTPS 443 [30].

- **Gestão Ativa de Alertas:** O Prometheus e o Grafana oferecem painéis analíticos visualmente sofisticados para *troubleshooting* reativo, contudo, a plataforma carece de *trigger rules*. A configuração do *Alertmanager* permitiria o envio proativo de mensagens (ex: Telegram ou E-mail corporativo) sempre que a VM atingisse picos críticos de consumo de CPU/RAM.

Conclui-se que o rigor na documentação de incidentes (ex: o problema de volume no Pi-hole, o socket do Promtail e o troubleshooting da DMZ da MEO) moldou o aspeto central deste trabalho prático, traduzindo as falhas momentâneas em valiosas referências pedagógicas no domínio da Engenharia de Redes.

Capítulo 6

Conclusões

6.1 Síntese do Trabalho Realizado

Este projeto teve como objetivo central demonstrar que é possível gerir uma infraestrutura de serviços de rede de forma totalmente automatizada, reproduzível e segura, utilizando exclusivamente ferramentas *open source*. O resultado final é uma plataforma robusta e funcional que corre numa máquina virtual Ubuntu Server 24.04 LTS, orquestrando um ecossistema completo de *containers* Docker: um servidor DNS (Pi-hole), um *reverse proxy* estrito (Nginx), uma VPN (WireGuard), uma nuvem privada corporativa (Nextcloud), um servidor de e-mail (Docker-Mailserver) e uma *stack* avançada de observabilidade (Prometheus, Node Exporter, Loki, Promtail e Grafana).

Do ponto de vista técnico, o projeto atingiu todos os seus objetivos e superou as perspetivas iniciais. A infraestrutura é definida integralmente em código (ficheiros YAML do Docker Compose e *Playbooks* Ansible), com segredos blindados pelo Ansible Vault e versionada no repositório do GitLab. A implantação automática a cada *commit* pela *pipeline* de CI/CD prova que qualquer falha catastrófica no servidor poderia ser recuperada em poucos minutos. Bastaria provisionar uma nova VM e executar o repositório, uma garantia de resiliência que, em infraestruturas geridas manualmente, simplesmente não existe.

6.2 Aprendizagens e Contribuições

O processo de implementação revelou que os maiores desafios técnicos raramente surgem da configuração dos serviços em si, mas sim das interações e restrições entre as diferentes camadas da arquitetura. Problemas como o *bridging* incorreto do VMware (Secção 4.2.3), os erros de indentação YAML no Netplan, o bloqueio do tráfego WireGuard pelo *router* de fronteira, a partilha do *socket* do Docker com o Promtail ou até a expansão em tempo real do volume LVM, ilustram bem esta realidade. Cada um destes incidentes exigiu um raciocínio estruturado de diagnóstico, eliminando hipóteses uma a uma até identificar a causa raiz.

A utilização do `tcpdump` para diagnosticar o problema de conectividade do WireGuard foi particularmente elucidativa: ao constatar que zero pacotes chegavam à interface de rede do servidor, foi possível descartar de imediato qualquer problema na configuração do *container* ou do protocolo, focando o diagnóstico na barreira de rede externa (a configuração de DMZ no *router*). Esta abordagem metódica — de observar e analisar o tráfego antes de agir — é um princípio fundamental da administração de sistemas que este projeto ajudou a consolidar na prática.

A integração do Ansible com a *pipeline* GitLab CI/CD demonstrou como a filosofia de *Infrastructure as Code* elimina a dicotomia histórica entre o que está documentado e o que está realmente instalado [1]. Neste modelo, o repositório Git é, simultaneamente, a documentação viva e o estado real da infraestrutura.

6.3 Comparação com Trabalhos Relacionados

A abordagem adotada neste projeto partilha a filosofia de orquestração leve com outras soluções documentadas na literatura e na comunidade técnica. Em comparação com arquiteturas baseadas em Kubernetes [2], a solução Docker Compose apresenta uma complexidade operacional consideravelmente menor e é mais adequada para ambientes de pequena escala, como o laboratório académico ou PMEs. Para contextos com dezenas ou

centenas de serviços e requisitos severos de *auto-scaling*, o Kubernetes seria a escolha natural, mas introduz uma curva de aprendizagem e uma sobrecarga de gestão de *hardware* que não se justificam neste cenário.

A escolha do WireGuard em detrimento do tradicional OpenVPN, fundamentada nas propriedades criptográficas modernas e na reduzida base de código apresentadas por Donenfeld [15], revelou-se extremamente acertada. A configuração foi significativamente mais simples, a integração com *Dynamic DNS* fluiu sem atritos e o desempenho do túnel foi visivelmente superior em testes subjetivos de latência e navegação remota.

6.4 Trabalho Futuro

Com base na análise crítica dos resultados (Capítulo 5) e considerando que os serviços principais e de segurança (Vault, Loki, E-mail e Nuvem) foram concluídos com sucesso, identificam-se as seguintes linhas de evolução natural para a plataforma:

- **Cifragem HTTPS com Let's Encrypt:** Implementar a terminação TLS real no Nginx utilizando certificados públicos gratuitos gerados pelo *Let's Encrypt* (via *certbot*). Em alternativa, migrar o encaminhamento web para o **Traefik**, que atua como *reverse proxy* com renovação automática de certificados [30].
- **Alertas Automáticos Proativos:** Configurar regras de notificação no *Alertmanager* do Grafana. O objetivo seria despoletar o envio automático de mensagens (por exemplo, via Telegram ou para o e-mail corporativo interno agora ativo) sempre que a utilização de CPU ultrapassasse os 85% ou a disponibilidade de memória RAM caísse para valores críticos [17].
- **Estratégia de Backups Off-site:** Uma vez que os dados do Nextcloud e as caixas de correio do Mailserver estão atualmente persistidos em volumes locais no host, a implementação de uma rotina automatizada de backup encriptado (utilizando ferramentas como o Restic ou o BorgBackup) para um armazenamento externo protegeria a infraestrutura contra falhas físicas do disco da máquina virtual.

6.5 Considerações Finais

Este projeto demonstrou que a conceção e a gestão profissional de infraestruturas de rede já não são um exclusivo de grandes organizações com largas equipas de TI. As ferramentas *open source* disponíveis atualmente — Docker, Ansible, GitLab, Prometheus, WireGuard — alcançaram um nível de maturidade e documentação que permite a um único administrador de sistemas, munido dos conhecimentos técnicos adequados, construir e operar uma plataforma de serviços robusta, cifrada e totalmente automatizada.

Fica provado que o principal obstáculo na transição para um *data center* moderno não é de cariz tecnológico, mas sim metodológico: passa por adotar desde o primeiro dia a disciplina inegociável do versionamento contínuo, da configuração declarativa e da documentação sistemática dos problemas encontrados.

Bibliografia

- [1] K. Morris, *Infrastructure as Code: Managing Servers in the Cloud*. O'Reilly Media, 2016, ISBN: 978-1-491-92435-8.
- [2] Docker Inc. «Docker Documentation,» acedido em 1 de mar. de 2026. URL: <https://docs.docker.com/>
- [3] J. Humble e D. Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley Professional, 2010, ISBN: 978-0-321-60191-9.
- [4] VMware Inc. «VMware Workstation Pro Documentation,» acedido em 28 de fev. de 2026. URL: <https://docs.vmware.com/en/VMware-Workstation-Pro/index.html>
- [5] IBM. «Containerization Explained,» acedido em 1 de mar. de 2026. URL: <https://www.ibm.com/topics/containerization>
- [6] Docker Inc. «Docker Compose Overview,» acedido em 1 de mar. de 2026. URL: <https://docs.docker.com/compose/>
- [7] DigitalOcean. «How To Install and Use Docker Compose on Ubuntu,» acedido em 1 de mar. de 2026. URL: <https://www.digitalocean.com/community/tutorials/how-to-install-and-use-docker-compose-on-ubuntu-20-04>
- [8] Red Hat. «Podman: A Tool for Managing OCI Containers and Pods,» acedido em 15 de mar. de 2026. URL: <https://podman.io/>

- [9] Red Hat. «Ansible Documentation,» acedido em 28 de fev. de 2026. URL: <https://docs.ansible.com/>
- [10] P. Mockapetris. «Domain Names — Implementation and Specification (RFC 1035),» Internet Engineering Task Force (IETF), acedido em 1 de mar. de 2026. URL: <https://www.rfc-editor.org/rfc/rfc1035>
- [11] Pi-hole LLC. «Pi-hole — Network-Wide Ad Blocking,» acedido em 1 de mar. de 2026. URL: <https://pi-hole.net/>
- [12] F5 Networks / Nginx Inc. «Nginx Documentation,» acedido em 1 de mar. de 2026. URL: <https://nginx.org/en/docs/>
- [13] B. Beyer, C. Jones, J. Petoff e N. R. Murphy, *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media, 2016, ISBN: 978-1-491-92912-4. URL: <https://sre.google/sre-book/table-of-contents/>
- [14] GitLab Inc. «GitLab CI/CD Documentation,» acedido em 21 de abr. de 2026. URL: <https://docs.gitlab.com/ee/ci/>
- [15] J. A. Donenfeld, «WireGuard: Next Generation Kernel Network Tunnel,» 2017. acedido em 30 de abr. de 2026. URL: <https://www.ndss-symposium.org/ndss2017/ndss-2017-programme/wireguard-next-generation-kernel-network-tunnel/>
- [16] Cloud Native Computing Foundation. «Prometheus Documentation,» acedido em 22 de abr. de 2026. URL: <https://prometheus.io/docs/introduction/overview/>
- [17] Grafana Labs. «Grafana Documentation,» acedido em 22 de abr. de 2026. URL: <https://grafana.com/docs/grafana/latest/>
- [18] M. Souppaya, J. Morello e K. Scarfone, «Application Container Security Guide,» National Institute of Standards e Technology (NIST), rel. téc. NIST SP 800-190, 2017. acedido em 22 de abr. de 2026. URL: <https://doi.org/10.6028/NIST.SP.800-190>
- [19] Canonical Ltd. «Ubuntu Server Documentation,» acedido em 28 de fev. de 2026. URL: <https://ubuntu.com/server/docs>

- [20] Canonical Ltd. «Netplan Documentation,» acedido em 28 de fev. de 2026. URL: <https://netplan.readthedocs.io/en/stable/>
- [21] Microsoft. «Windows Subsystem for Linux Documentation,» acedido em 28 de fev. de 2026. URL: <https://learn.microsoft.com/en-us/windows/wsl/>
- [22] Docker Inc. «Docker Security Best Practices,» acedido em 21 de abr. de 2026. URL: <https://docs.docker.com/engine/security/>
- [23] Pi-hole LLC. «Pi-hole Docker Documentation,» acedido em 1 de mar. de 2026. URL: <https://github.com/pi-hole/docker-pi-hole>
- [24] NGINX Inc. «What Is a Reverse Proxy Server?» Acedido em 21 de abr. de 2026. URL: <https://www.nginx.com/resources/glossary/reverse-proxy-server/>
- [25] LinuxServer.io. «LinuxServer WireGuard Docker Image Documentation,» acedido em 30 de abr. de 2026. URL: <https://docs.linuxserver.io/images/docker-wireguard/>
- [26] Cloudflare. «What is a DMZ Network?» Acedido em 30 de abr. de 2026. URL: <https://www.cloudflare.com/learning/network-layer/what-is-a-dmz-network/>
- [27] GitLab Inc. «GitLab Runner Documentation,» acedido em 21 de abr. de 2026. URL: <https://docs.gitlab.com/runner/>
- [28] TechWorld with Nana. «GitLab CI CD Pipeline Tutorial Including Deployment to a Server.» Video YouTube, acedido em 21 de abr. de 2026. URL: <https://www.youtube.com/watch?v=qP8kir2GUgo>
- [29] Grafana Labs. «Node Exporter Full — Grafana Dashboard 1860,» acedido em 22 de abr. de 2026. URL: <https://grafana.com/grafana/dashboards/1860-node-exporter-full/>
- [30] E. Rescorla. «The Transport Layer Security (TLS) Protocol Version 1.3 (RFC 8446),» Internet Engineering Task Force (IETF), acedido em 29 de abr. de 2026. URL: <https://www.rfc-editor.org/rfc/rfc8446>

Apêndice A

Proposta Original do Projeto

Plataforma de serviços de rede baseada em containers

Inteligência artificial	Multimédia/Realidade aumentada/...
Aplicações móveis	x Redes e gestão de sistemas
Plataformas web	*

* - A especificar pelo proponente

Orientador: Nuno Rodrigues

Coorientador: Eduardo Costa

1 Objetivo

- Construir uma plataforma de serviços de rede (DNS, web, reverse proxy, e-mail, VPN, etc.) baseada em containers e orquestração leve, focada em boas práticas de segurança, CI/CD e observabilidade.
- Demonstrar como um pequeno “data center” de serviços pode ser gerido de forma reproduzível usando apenas ferramentas open source.

2 Detalhes

O projeto deve definir um conjunto de serviços de rede típicos de uma infraestrutura de pequena/média dimensão (ex.: servidor DNS recursivo/autoritativo, reverse proxy HTTPS, serviços web e e-mail, servidor de ficheiros, VPN).

Todos os serviços devem ser implementados em containers (Docker/Podman) com definição declarativa (compose ou similar), favorecendo configurações reproduzíveis e com versionamento. Devem ser integrados mecanismos de observabilidade e monitorização open source (ex.: Prometheus, Grafana, Loki, ELK/EFK) para recolha de métricas, logs centralizados e alertas básicos. O trabalho a desenvolver focar-se-á também na aplicação de boas práticas de segurança: princípio do mínimo privilégio nos containers, gestão de segredos, TLS, atualização automatizada de imagens, hardening básico de serviços.

O projeto deve contemplar uma pipeline simples de CI/CD (ex.: com Git e um runner local), permitindo testar alterações de configuração e fazer deploy controlado para o ambiente de produção de laboratório. O resultado final deverá incluir um “runbook” de administrador, com procedimentos para deploy, rollback,

troubleshooting e recuperação de falhas dos serviços.

3 Metodologia de trabalho

- Levantamento das necessidades funcionais (serviços a disponibilizar) e seleção das imagens/ferramentas open source adequadas.
- Modelação da arquitetura lógica (rede interna de containers, exposição para o exterior, política de firewall) e implementação incremental com testes unitários a cada serviço.
- Integração progressiva de módulos de observabilidade e monitorização, configurando dashboards e alertas mínimos para indicadores chave (latência, erros 5xx, saturação de CPU/RAM, etc.).
- Implementação de uma pipeline simples de CI/CD, testes de atualização e falha e elaboração de documentação completa (arquitetura, scripts, procedimentos).

Dimensão da equipa:	1 a 2 elementos
Recursos necessários:	PC com suporte de virtualização

Apêndice B

Listagens de Código Fonte e Configurações

Neste apêndice encontram-se listados os principais ficheiros de configuração e automação desenvolvidos e referenciados ao longo deste relatório para o aprovisionamento da infraestrutura.

B.1 Docker Compose da Plataforma (`docker-compose.yml`)

```
version: '3.8'

services:
  # REVERSE PROXY E SERVIDOR WEB (Nginx)
  proxy_web:
    image: nginx:latest
    container_name: nginx_proxy
    restart: unless-stopped
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - ./web:/usr/share/nginx/html:ro
```

```

- ./nginx/default.conf:/etc/nginx/conf.d/default.conf:ro
networks:
- rede_interna

# SERVIDOR DNS (Pi-Hole)
servidor_dns:
  image: pihole/pihole:latest
  container_name: dns_pihole
  restart: unless-stopped
  ports:
    - "53:53/tcp"
    - "53:53/udp"
    - "8080:80"
  environment:
    TZ: 'Europe/Lisbon'
    WEBPASSWORD: "{{_vault_pihole_password_}}"
  networks:
    - rede_interna

# REDE PRIVADA VIRTUAL (WireGuard VPN)
wireguard:
  image: lscr.io/linuxserver/wireguard:latest
  container_name: wireguard
  cap_add:
    - NET_ADMIN
    - SYS_MODULE
  environment:
    - PUID=1000
    - PGID=1000
    - TZ=Europe/Lisbon
    - SERVERURL=daniel-infra.ddns.net
    - SERVERPORT=51821
    - PEERS=3
    - PEERDNS=192.168.1.252
    - INTERNAL_SUBNET=10.13.13.0

```

```

    - ALLOWEDIPS=0.0.0.0/0
volumes:
    - ./wireguard:/config
ports:
    - "51821:51821/udp"
sysctls:
    - net.ipv4.conf.all.src_valid_mark=1
restart: unless-stopped
networks:
    - rede_interna

# SERVIDOR DE FICHEIROS / CLOUD PRIVADA (Nextcloud)
nextcloud:
    image: lscr.io/linuxserver/nextcloud:latest
    container_name: nextcloud
    environment:
        - PUID=1000
        - PGID=1000
        - TZ=Europe/Lisbon
    volumes:
        - ./nextcloud/appdata:/config
        - ./nextcloud/data:/data
    ports:
        - "8081:443"
    restart: unless-stopped
    networks:
        - rede_interna

# SERVIDOR DE E-MAIL (Docker-Mailserver)
mailserver:
    image: mailserver/docker-mailserver:latest
    container_name: mailserver
    hostname: mail
    domainname: local.lan
    ports:

```

```

- "25:25"
- "143:143"
- "587:587"
- "993:993"
volumes:
- ./mailserver/maildata:/var/mail
- ./mailserver/mailstate:/var/mail-state
- ./mailserver/mailconfig:/tmp/docker-mailserver
- /etc/localtime:/etc/localtime:ro
environment:
- ENABLE_SPAMASSASSIN=1
- ENABLE_CLAMAV=0
- ENABLE_FAIL2BAN=1
- ONE_DIR=1
restart: unless-stopped
networks:
- rede_interna

# Node Exporter (Metricas do Host)
node-exporter:
  image: prom/node-exporter:latest
  container_name: node_exporter
  restart: unless-stopped
  ports:
    - "9100:9100"
  networks:
    - rede_interna

# Prometheus (Base de dados de metricas)
prometheus:
  image: prom/prometheus:latest
  container_name: prometheus
  restart: unless-stopped
  ports:
    - "9090:9090"

```

```

volumes:
  - ./prometheus/prometheus.yml:/etc/prometheus/prometheus.yml:ro
networks:
  - rede_interna

# Grafana (Visualizacao de Dashboards)
grafana:
  image: grafana/grafana:latest
  container_name: grafana
  restart: unless-stopped
  ports:
    - "3000:3000"
  environment:
    - GF_SECURITY_ADMIN_PASSWORD={{ vault_grafana_password }}
    - GF_SERVER_ROOT_URL=http://192.168.1.252/grafana/
    - GF_SERVER_SERVE_FROM_SUB_PATH=true
  volumes:
    - ./grafana/provisioning:/etc/grafana/provisioning
  networks:
    - rede_interna

# Grafana Loki (Agregador de Logs)
loki:
  image: grafana/loki:3.0.0
  container_name: loki
  ports:
    - "3100:3100"
  restart: unless-stopped
  networks:
    - rede_interna

# Promtail (Agente de recolha de logs)
promtail:
  image: grafana/promtail:3.0.0
  container_name: promtail

```

```

volumes:
  - /var/log:/var/log:ro
  - /var/lib/docker/containers:/var/lib/docker/containers:ro
  - /var/run/docker.sock:/var/run/docker.sock:ro
  - ./promtail/promtail-config.yaml:/etc/promtail/config.yaml:ro
command: -config.file=/etc/promtail/config.yaml
restart: unless-stopped
networks:
  - rede_interna

networks:
  rede_interna:
    driver: bridge

```

B.2 Ansible Playbook de Setup (setup_docker.yml)

```

---
- name: Configuracao inicial do servidor e instalacao do Docker
  hosts: servidores
  become: yes

  tasks:
    - name: Atualizar a cache do apt
      apt:
        update_cache: yes
        cache_valid_time: 3600

    - name: Instalar pacotes pre-requisitos do sistema
      apt:
        name:
          - apt-transport-https
          - ca-certificates
          - curl

```

```

        - software-properties-common
    state: present

- name: Instalar o Docker e o Docker Compose
  apt:
    name:
      - docker.io
      - docker-compose-v2
    state: present

- name: Garantir que o serviço docker arranque
  systemd:
    name: docker
    state: started
    enabled: yes

- name: Adicionar o utilizador ao grupo docker
  user:
    name: "{{ ansible_user }}"
    groups: docker
    append: yes

```

B.3 Ansible Playbook de Deploy (deploy_servicos.yml)

```

---
- name: Deploy da Stack de Serviços de Rede
  hosts: localhost
  connection: local
  become: yes

  vars_files:
    - vars/secrets.yml

```

```

tasks:
  - name: Garantir que a pasta de destino existe
    ansible.builtin.file:
      path: /opt/plataforma_servicos
      state: directory
      mode: '0755'

  - name: Copiar pastas de configuracao essenciais
    ansible.builtin.copy:
      src: "../servicos/{{_item}}"
      dest: /opt/plataforma_servicos/
      mode: '0755'
    loop:
      - nginx
      - prometheus
      - promtail
      - grafana
      - web

  - name: Copiar e configurar o docker-compose com Vault
    ansible.builtin.template:
      src: ../servicos/docker-compose.yml
      dest: /opt/plataforma_servicos/docker-compose.yml
      mode: '0600'

  - name: Executar o Docker Compose
    community.docker.docker_compose_v2:
      project_src: /opt/plataforma_servicos
      state: present
      pull: missing

```

B.4 GitLab CI/CD Pipeline (.gitlab-ci.yml)


```
stages:
  - deploy

deploy_infraestrutura:
  stage: deploy
  tags:
    - meu-runner-local
  script:
    - echo "A iniciar o deploy com segredos Vault..."
    - cd infra
    - echo "$ANSIBLE_VAULT_PASSWORD" > .vault_key
    - sudo ansible-playbook -i "localhost," -c local --vault-password-
      file .vault_key deploy_servicos.yml
    - rm -f .vault_key
    - echo "Deploy concluido com sucesso!"
```